

Mech-Exosuit Framework

[Mod Name]

Exosuit meccanoid-guidate con subcore vanilla come pilota interno. Integrazione di Exosuit Framework, vanilla Mechtech e SubcoreInfo.

01 Subcore Pilota Standard / High subcore installato fisicamente come body part interna, con HP separati	02 Bay Ibrida 4 funzioni: loadout, gestazione, charging, subcore install/extract	03 Relitto Sigillato Drop deterministico basato su HP residui del subcore al momento della morte
---	---	---

v0.1 Draft

RimWorld 1.6

Exosuit Framework

SubcoreInfo (soft dep)

Combat Extended compat

Indice

1. Concept & Vision	2
2. Architettura Tecnica	5
3. Schema Defs XML	17
4. Sistema di Danno & Morte	33
5. Bay Ibrida & UI	39
6. Batteria & Bandwidth	44
7. Integrazione SubcoreInfo & Combat Extended	47
8. Bilanciamento — Caso Pilota Patriot	51
9. Albero Tecnologico	56
10. Roadmap MVP	58
11. Checklist Implementazione	62
12. Glossario	66

CAPITOLO 01 · CONCEPT

1. Concept & Vision

[Mod Name] è una mod per RimWorld 1.6 che introduce exosuit meccanoid-guidate: entità ibride che combinano l'estetica e l'autonomia operativa dei mechanoidi vanilla con la modularità e la configurabilità dell'Exosuit Framework di Aoba. Il cuore del concept è semplice ma tematicamente potente: invece di un pawn umano a bordo dell'exosuit, il pilota è un **subcore vanilla** (Standard o High) installato fisicamente come body part interna, con health pool separato dal mech.

L'idea nasce dall'osservazione che le exosuit classiche — pur essendo visivamente e meccanicamente soddisfacenti — soffrono di due limiti: (1) richiedono un pawn umano dedicato che viene 'bloccato' all'interno, riducendo la flessibilità della colonia; (2) il flavor tematico è quello di un'veicolo con pilota' anziché di un'autonomia robotica. D'altra parte, i mechanoidi vanilla hanno autonomia ma sono monolitici: niente slot di equipaggiamento, niente loadout configurabile. [Mod Name] unisce i due mondi: il subcore diventa letteralmente il 'cervello' del mech-exosuit, con tutti i pro e i contro che questo comporta.

Il posizionamento della mod è quindi quello di un framework estensibile che può 'mech-ificare' qualsiasi exosuit esistente (compatibile con Exosuit Framework) aggiungendo un nuovo ThingComp e una nuova bay di gestione. La mod è pensata come **soft dependency**: funziona standalone ma si arricchisce quando SubcoreInfo è installato (preserva l'identità del pawn scansionato) e quando Combat Extended è presente (usa il suo sistema di penetrazione armatura).

Tabella 1.1 — Feature chiave della mod

Aspetto	Decisione	Note implementative
Pilota interno	Subcore Standard/High vanilla	BodyDef custom con subcore come body part, HP 100/175
Danno al pilota	Health pool separato, penetrazione come exosuit vanilla	Coverage 0.10 (10% dei colpi che penetrano l'armatura colpiscono il subcore)
Morte subcore	Deterministica	HP subcore = 0 → Destroyed Subcore (smontabile al machining table)
Morte mech	Drop Relitto Sigillato	Apertura deterministica basata su HP residui del subcore

Aspetto	Decisione	Note implementative
Batteria	Scarsa (~mezza giornata)	Solo bay ibrida ricarica; vanilla/Deadmanswitch charger disattivati
Bandwidth	Come mech vanilla	Entra in control group del mechanitor, va dormant se offline
Sblocco tech	Standard Mechtech prerequisito	No Basic subcore — solo Standard/High per coerenza lore
Skill	Suit base + bonus subcore	Standard: +0; High: +4 a tutte le skill della suit
Estrazione	Comando floating menu	Solo mech dockato in bay → restituisce subcore + frame item
SubcoreInfo	Soft dependency	CopySubcoreInfo() su installazione e su recupero da relitto

Tabella 1.1 — Riepilogo delle decisioni di design core della mod.

1.1 — Pilastri di design

Tre pilastri guidano tutte le decisioni di design e bilanciamento della mod. Ogni feature proposta deve essere valutata rispetto a questi pilastri; se una feature li viola, va rivista o scartata.

Pilastro 1 — Costo reale al fallimento

In molte mod di mech/exosuit, la distruzione del mech comporta solo la perdita di materiali. Questo genera gameplay superficiale: si spamma mech senza cura perché il costo di una morte è solo 'craft un altro'. [Mod Name] rompe questo pattern rendendo il subcore un oggetto **persistente e fragile**: un subcore può sopravvivere a più morti del mech, ma ogni volta perde HP e diventa più probabile che venga distrutto al prossimo blow-through. Questo crea una progressione naturale di 'subcore storici' nella colonia — alcuni diventano reliquie da proteggere, altri finiscono per essere smontati quando troppo danneggiati per essere utili.

Pilastro 2 — Tensione operativa

La batteria scarsa non è un limite arbitrario: è una scelta di game design per creare tensione. Il giocatore non può lasciare un mech-exosuit sempre attivo a pattugliare — deve pianificare quando attivarlo, quando ricaricarlo, e questo crea finestre di vulnerabilità narrative. La bay ibrida diventa un asset strategico: posizionarla vicino al perimetro difensivo o al centro della base cambia le dinamiche di risposta agli raid. Inoltre, il fatto che consumi bandwidth anche da spento impedisce di avere una scorta di

mech-exosuit pronti all'uso — ogni mech attivo ha un costo opportunità reale.

Pilastro 3 — Rispetto delle mod esistenti

[Mod Name] non riscrive Exosuit Framework né vanilla mechtech: ne estende il comportamento via ThingComp, ModExtension e Harmony patch non-invasive. Una suit 'mech-ificata' è, dal punto di vista di Exosuit Framework, una normale exosuit con un Comp aggiuntivo. Dal punto di vista di vanilla mechtech, è un mech normale in un control group del mechanitor. Questo garantisce massima compatibilità con future versioni delle mod dipendenti e minimizza il rischio di regressioni.

1.2 — Posizionamento rispetto alle mod esistenti

Per capire dove si posiziona [Mod Name], è utile mappare lo spazio delle mod di mech/exosuit esistenti per RimWorld 1.6:

Mod	Pilota	Batteria	Configurabilità	Costo morte
Vanilla Mechanoids	Nessuno (auto nomo)	Ricarica standard	Bassa (preset)	Solo materiali
Exosuit Framework	Pawn umano	Fuel cell opzionale	Alta (slot system)	Pawn + materiali
TheDeadmanswitch Mech Chargers	N/A (framework)	Custom charger	N/A	N/A
Androids / Misc Robots	AI core	Ricarica standard	Media	AI core perso
[Mod Name]	Subcore vanilla	Scarsa (batteria ibrida)	Alta (slot system)	Subcore persistente con degrado

Tabella 1.2 — Posizionamento competitivo. [Mod Name] occupa la nicchia "mech autonomi con subcore persistente e loadout configurabile".

1.3 — Esperienza target

Il giocatore tipico a cui ci rivolgiamo è il **colonnello mech-friendly**: ha già sbloccato Standard Mechtech, ha un mechanitor attivo con qualche lifter/constructoid, e sta cercando un'evoluzione che unisca la potenza difensiva delle exosuit (armatura, slot di armi pesanti) con l'autonomia dei mech (non richiedere un pawn dedicato). [Mod Name] dovrebbe dargli:

- **Una decisione tattica significativa** quando attivare il mech-exosuit (batteria scarica = mech inutilizzabile per ore)

- **Un legame emotivo con i subcore** — specialmente se SubcoreInfo è installato e mostra il nome del pawn scansionato
- **Una perdita che fa male** quando un subcore storico viene distrutto (ma non così male da scoraggiare il retry)
- **Una progression lenta ma tangibile** — più subcore crafti, più mech-exosuit puoi fieldare, ma bandwidth limita il numero contemporaneo
- **Compatibilità con il loadout** delle sue exosuit preferite (Patriot, AMP, ecc.) — la mod non lo costringe a imparare nuove suit

Nota su sottosistema SubcoreInfo

L'integrazione con SubcoreInfo è opzionale ma fortemente raccomandata. Senza di essa, il subcore è "solo un oggetto". Con SubcoreInfo, diventa il ricordo persistente di un pawn — possibilmente un pawn morto per produrlo (ripguard). Questo aggiunge peso emotivo al sistema di degrado: ogni riparazione fallita non è solo una perdita meccanica, è la perdita dell'ultimo legame con quel pawn. Implementare l'integrazione costa ~20 righe di codice (vedi Sezione 7).

CAPITOLO 02 · ARCHITETTURA

2. Architettura Tecnica

L'architettura di [Mod Name] si compone di **tre layer logici**: (1) un layer di comportamenti runtime, costituito dai ThingComp e dalle Building custom che gestiscono subcore, batteria e UI; (2) un layer di Defs XML statiche che descrivono i nuovi oggetti (mech-exosuit, relitto, subcore distrutto, bay ibrida); (3) un layer di integrazione con API esterne — Exosuit Framework, SubcoreInfo e vanilla mechtch — che vengono consumate in sola lettura via Harmony patch e reflection sicura.

Il diagramma sottostante illustra le classi C# principali e le loro relazioni. Le classi nuove (cinque in totale) sono evidenziate in rame con bordo sinistro rinforzato. Le classi di Exosuit Framework e SubcoreInfo sono mostrate in blu tratteggiato per indicare che sono dipendenze esterne consumate via interfaccia o reflection.

2.1 — Diagramma dell'architettura

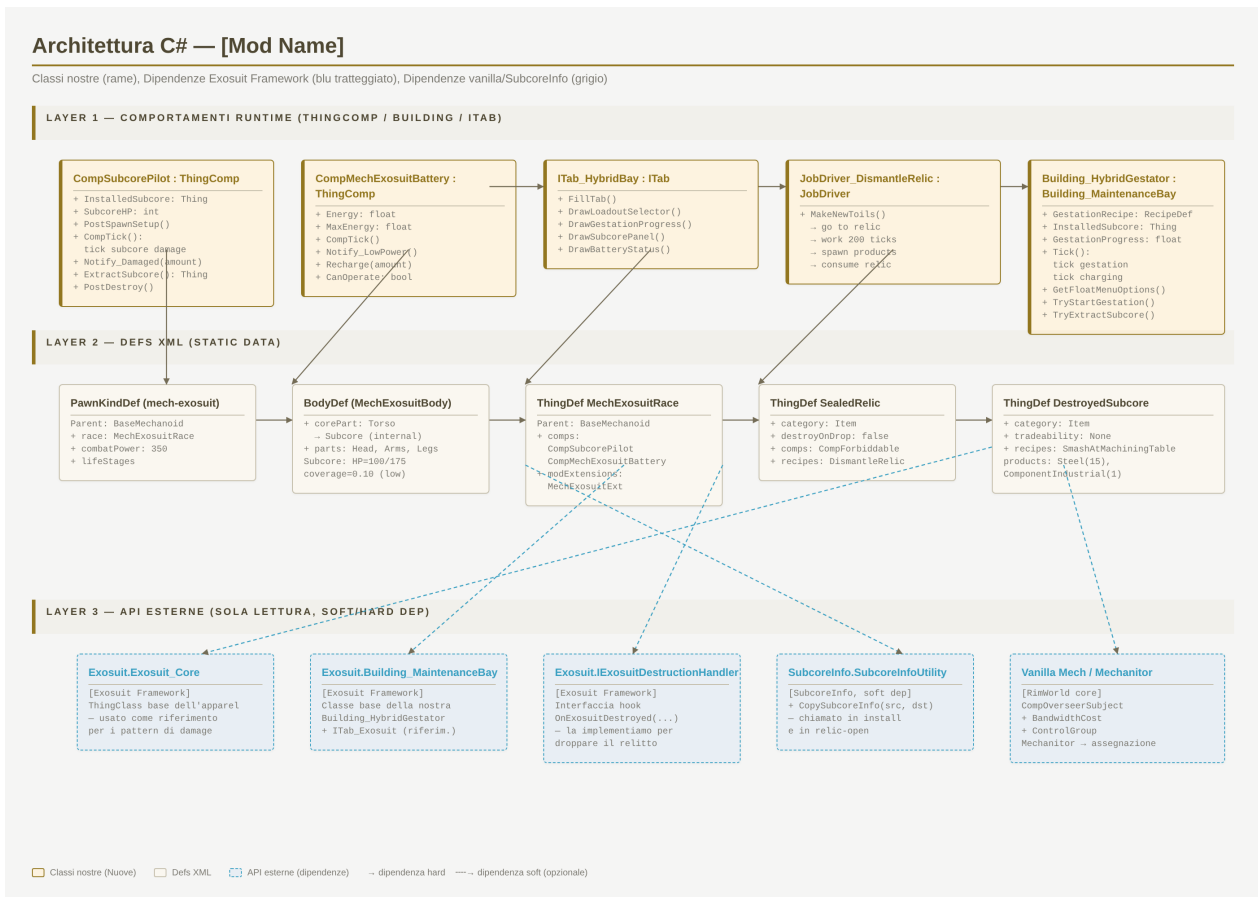


Figura 2.1 — Diagramma delle classi C# e delle loro dipendenze. Layer 1: ThingComp/Building runtime. Layer 2: Defs XML statiche. Layer 3: API esterne.

2.2 — Classi C# principali

Di seguito gli scheletri delle cinque classi C# principali. Per ciascuna sono mostrate le firme pubbliche e l'implementazione di 1-2 metodi chiave; il resto è marcato come // TODO e verrà completato in fase di coding. La convenzione di naming segue quella di RimWorld (PascalCase per metodi pubblici, camelCase per campi privati, underscore per campi serializzati).

2.2.1 — CompSubcorePilot

Il Comp centrale della mod. Si attacca a ogni mech-exosuit via XML (). Gestisce: installazione del subcore all'atto della gestazione, tracking dell'HP del subcore (separato dall'HP del mech), danneggiamento su blow-through, estrazione volontaria, e drop del relitto su morte.

```
// CompSubcorePilot.cs — scheletro

namespace ModName
{
    public class CompSubcorePilot : ThingComp
    {

```

```
// Stato serializzato (salvato nel savegame)

private Thing installedSubcore;      // ref al subcore installato (o null)
private int subcoreHP;              // HP corrente del subcore
private int subcoreMaxHP;           // max HP in base al tier (100/175)

public Thing InstalledSubcore => installedSubcore;
public int SubcoreHP => subcoreHP;
public int SubcoreMaxHP => subcoreMaxHP;
public bool HasSubcore => installedSubcore != null;

public override void PostExposeData()
{
    Scribe_References.Look(ref installedSubcore, "installedSubcore");
    Scribe_Values.Look(ref subcoreHP, "subcoreHP", 0);
    Scribe_Values.Look(ref subcoreMaxHP, "subcoreMaxHP", 100);
}

// Chiamato quando il mech-exosuit viene generato dalla bay
public void InstallSubcore(Thing subcoreItem)
{
    // TODO: validazione (tier corretto, non già installato)

    installedSubcore = subcoreItem;
    subcoreMaxHP = (subcoreItem.def == ThingDefOf.SubcoreHigh) ? 175 : 100;
    subcoreHP = subcoreMaxHP;

    // Integrazione SubcoreInfo (soft dep, via reflection)
    SubcoreInfoBridge.CopySubcoreInfo(subcoreItem, parent);
}

// Chiamato da Harmony patch su Pawn.TakeDamage quando il colpo
// penetra e colpisce la body part "Subcore"
public void Notify_SubcoreHit(DamageInfo dinfo)
{
    subcoreHP -= dinfo.Amount;
```

```
        if (subcoreHP <= 0)
        {
            OnSubcoreDestroyed();
        }
    }

    private void OnSubcoreDestroyed()
    {
        // TODO: spawn "Destroyed [Std/High] Subcore" come item separato
        // TODO: mettere il mech in stato "braindead" (downed, non riparabile)
        // TODO: notifica messaggio al giocatore

        installedSubcore = null;
        subcoreHP = 0;
    }

    // Chiamato quando il mech muore (HP mech = 0)
    public void OnMechDeath()
    {
        if (installedSubcore == null) return;

        // TODO: determinare se subcore sopravvive in base a subcoreHP
        // TODO: spawn Relitto Sigillato con subcore "intrappolato" dentro
        // (il relitto ha un CompRelic che conserva HP residui + identità)
    }

    // Comando floating menu: estrai subcore (richiede mech in bay)
    public void ExtractSubcore(Building_HybridGestator bay)
    {
        // TODO: validare che il mech sia dockato
        // TODO: spawn subcore item accanto alla bay
        // TODO: SubcoreInfoBridge.CopySubcoreInfo(parent, subcoreItem) per preservare identità
        // TODO: disattivare il mech (diventa "shell vuota" da ricaricare)

        installedSubcore = null;
        subcoreHP = 0;
    }
```

```

    }

    public class CompProperties_SubcorePilot : CompProperties
    {
        public CompProperties_SubcorePilot()
        {
            compClass = typeof(CompSubcorePilot);
        }
    }
}

```

2.2.2 — CompMechExosuitBattery

Gestisce l'energia del mech-exosuit. Diversamente dal FuelCell di Exosuit Framework (che è un boost opzionale), questo Comp è la **fonte primaria di operatività**: quando l'energia arriva a 0, il mech va in 'low power mode' (downed, non morto). La ricarica avviene solo nella bay ibrida — i charger vanilla e di TheDeadmanswitch sono disabilitati per questi mech via Patch.

```

// CompMechExosuitBattery.cs — scheletro
namespace ModName
{
    public class CompMechExosuitBattery : ThingComp
    {
        private float energy;          // 0..1 (normalizzato)

        private float maxEnergyTicks;  // ~12h gioco = 36000 ticks per Standard; 18h = 54000 per High

        public float EnergyPct => energy;

        public bool CanOperate => energy > 0.05f;

        public bool IsLowPower => energy < 0.10f;

        public override void CompTick()
        {
            if (!CanOperate) return;

            energy -= 1f / maxEnergyTicks;

            if (energy < 0) energy = 0;
        }
    }
}

```

```
        if (energy == 0 && !IsInLowPowerState)
        {
            Notify_LowPower();
        }
    }

    public void Notify_LowPower()
    {
        // TODO: downed state (non death) — come mech senza bandwidth
        // TODO: messaggio "Mech-Exosuit [X] è andato in low power mode"
        // TODO: gizmo "Ritorna alla bay" auto-attivato
    }

    public void Recharge(float amount)
    {
        energy = System.Math.Min(1f, energy + amount);
    }

    public override void PostExposeData()
    {
        Scribe_Values.Look(ref energy, "energy", 1f);
        Scribe_Values.Look(ref maxEnergyTicks, "maxEnergyTicks", 36000f);
    }

    public override string CompInspectStringExtra()
    {
        return $"Batteria: {energy*100:F0}%\nStato: {(CanOperate ? "Operativo" : "Low power")}";
    }
}

public class CompProperties_MechExosuitBattery : CompProperties
{
    public float maxEnergyTicks = 36000f; // override da XML per tier diverso
}
```

```

        public CompProperties_MechExosuitBattery()
        {
            compClass = typeof(CompMechExosuitBattery);
        }
    }
}

```

2.2.3 — Building_HybridGestator

Estende Exosuit.Building_MaintenanceBay aggiungendo quattro funzioni: (1) loadout UI stile ITab_Exosuit, (2) gestazione del mech a partire da subcore + materiali, (3) stazione di ricarica per mech dockato, (4) comandi floating per installazione/estrazione subcore. Vedi Sezione 5 per il dettaglio della UI.

```

// Building_HybridGestator.cs — scheletro
namespace ModName
{
    public class Building_HybridGestator : Exosuit.Building_MaintenanceBay
    {
        private Thing pendingSubcore;      // subcore caricato per la prossima gestazione
        private float gestationProgress;    // 0..1
        private RecipeDef activeRecipe;
        private int chargingMechId = -1;    // ID del mech dockato per ricarica

        public bool IsGestating => activeRecipe != null;
        public float GestationPct => gestationProgress;

        public override void Tick()
        {
            base.Tick(); // eredita maintenance bay behavior

            // Tick gestazione
            if (IsGestating)
            {
                gestationProgress += 1f / activeRecipe.workAmount;
                if (gestationProgress >= 1f)
                {

```

```
        CompleteGestation();
    }
}

// Tick charging per mech dockato
if (chargingMechId != -1)
{
    var mech = FindMechById(chargingMechId);
    if (mech?.TryGetComp() is { } batt)
    {
        batt.Recharge(0.001f); // 5W/tick ≈ ricarica ~3h game per 0->100%
    }
    else
    {
        chargingMechId = -1;
    }
}

public void TryStartGestation(RecipeDef recipe, Thing subcore)
{
    // TODO: validare materiali + skill + subcore tier
    // TODO: consumare materiali
    pendingSubcore = subcore;
    activeRecipe = recipe;
    gestationProgress = 0f;
}

private void CompleteGestation()
{
    // TODO: spawn Pawn (MechExosuitRace) usando PawnGenerator
    // TODO: InstallSubcore(pendingSubcore) sul mech appena creato
    // TODO: assegnazione al control group del mechanitor
    // TODO: notifica "Mech-Exosuit [X] è operativo"
```

```

        activeRecipe = null;
        gestationProgress = 0f;
        pendingSubcore = null;
    }

    public override IEnumerable GetFloatMenuOptions(Pawn pawn)
    {
        foreach (var opt in base.GetFloatMenuOptions(pawn))
            yield return opt;

        // TODO: aggiungi "Installa subcore" se mech vuoto e pawn ha subcore in inventory
        // TODO: aggiungi "Estrai subcore" se mech dockato con subcore
        // TODO: aggiungi "Avvia gestazione" se mech vuoto + materiali disponibili
    }
}

```

2.2.4 — ITab_HybridBay

Tab custom che replica la UI di ITab_Exosuit di Exosuit Framework per la selezione del loadout (frame + armi + moduli) e aggiunge pannelli per gestazione e stato batteria. Vedi Sezione 5 per dettaglio della UI.

```

// ITab_HybridBay.cs — scheletro
namespace ModName
{
    public class ITab_HybridBay : ITab
    {
        private Vector2 scrollPos = Vector2.zero;
        private Building_HybridGestator Bay => (Building_HybridGestator)SelThing;

        public ITab_HybridBay()
        {
            size = new Vector2(440f, 540f);
            labelKey = "TabHybridBay";
        }
    }
}

```

```

protected override void FillTab()
{
    // TODO: disegna 4 sezioni:
    // 1. Header con nome bay + stato (idle/gestating/charging)
    // 2. Loadout selector (replica ITab_Exosuit)
    // 3. Subcore panel (mostra subcore installato, HP, identità)
    // 4. Battery status (barra energia del mech dockato)
    // 5. Gestation progress bar (se in corso)

    var rect = new Rect(0, 0, size.x, size.y).ContractedBy(10f);
    Widgets.BeginScrollView(rect, ref scrollPos, viewRect);

    // ... rendering ...

    Widgets.EndScrollView();
}
}
}

```

2.2.5 — JobDriver_DismantleRelic

Job driver per lo smontaggio del Relitto Sigillato. Il pawn va al relitto, lavora per ~200 ticks, e produce: subcore recuperato (se gli HP residui lo permettono) o Destroyed Subcore + materiali.

```

// JobDriver_DismantleRelic.cs — scheletro
namespace ModName
{
    public class JobDriver_DismantleRelic : JobDriver
    {
        private const int WorkTicks = 200;

        private CompRelic Relic => job.targetA.Thing.TryGetComp();

        public override bool TryMakePreToilReservations(bool errorOnFailed)
        {
            return pawn.Reserve(job.targetA, job, 1, -1, null, errorOnFailed);
        }
    }
}

```

```
protected override IEnumerable MakeNewToils()
{
    this.FailOnDespawnedNullOrForbidden(TargetIndex.A);

    yield return Toils_Goto.GotoThing(TargetIndex.A, PathEndMode.Touch);
    yield return Toils_General.Wait(WorkTicks)
        .WithProgressBarToilDelay(TargetIndex.A)
        .FailOnDespawnedNullOrForbidden(TargetIndex.A);

    var openToil = new Toil();
    openToil.initAction = () =>
    {
        var relic = job.targetA.Thing;
        var comp = Relic;
        if (comp == null) return;

        // Determina esito in base a HP residui
        bool survives = comp.CalculateSurvival();
        if (survives)
        {
            var subcore = ThingMaker.MakeThing(comp.SubcoreDef);
            subcore.HitPoints = comp.CalculateRecoveredHP();
            SubcoreInfoBridge.CopySubcoreInfo(relic, subcore);
            GenPlace.TryPlaceThing(subcore, pawn.Position, Map, ThingPlaceMode.Near);
        }
        else
        {
            var destroyed = ThingMaker.MakeThing(comp.DestroyedSubcoreDef);
            GenPlace.TryPlaceThing(destroyed, pawn.Position, Map, ThingPlaceMode.Near);
        }

        // Materiali del relitto (sempre)
        foreach (var thingDef in comp.MaterialYield)
```

```

        {
            var mat = ThingMaker.MakeThing(thingDef.def);
            mat.stackCount = thingDef.count;
            GenPlace.TryPlaceThing(mat, pawn.Position, Map, ThingPlaceMode.Near);
        }

        relic.Destroy();
    };
    yield return openToil;
}
}
}

```

2.3 — SubcoreInfoBridge (helper per soft dependency)

Per mantenere [Mod Name] standalone pur supportando SubcoreInfo, tutte le chiamate a SubcoreInfo passano per un **bridge statico** che rileva a runtime se la mod è caricata (via LoadedModManager.RunningMods) e in caso contrario esegue un no-op. Questo pattern è idiomatico in RimWorld e permette alla mod di funzionare anche se SubcoreInfo viene disinstallato a metà savegame.

```

// SubcoreInfoBridge.cs — soft dependency wrapper
namespace ModName
{
    public static class SubcoreInfoBridge
    {
        private static bool? _isLoading;

        public static bool IsLoaded => _isLoading ??= IsModLoaded("eth0net.SubcoreInfo");

        private static bool IsModLoaded(string packageId)
        {
            return LoadedModManager.RunningMods
                .Any(m => m.PackageIdPlayerFacing == packageId);
        }
    }

    ///

```

```
/// Copia l'identità del pawn scansionato da subcore-item a mech-pawn
/// (o viceversa). No-op se SubcoreInfo non è caricato.
///
public static void CopySubcoreInfo(Thing source, Thing dest)
{
    if (!IsLoaded) return;
    try
    {
        // Reflection sicura: cerca il metodo pubblico CopySubcoreInfo in SubcoreInfo.dll
        var asm = AppDomain.CurrentDomain.GetAssemblies()
            .FirstOrDefault(a => a.GetName().Name == "SubcoreInfo");
        var util = asm?.GetType("SubcoreInfo.SubcoreInfoUtility");
        var method = util?.GetMethod("CopySubcoreInfo",
            new[] { typeof(Thing), typeof(Thing) });
        method?.Invoke(null, new object[] { source, dest });
    }
    catch (System.Exception e)
    {
        Log.Warning($"[ModName] SubcoreInfo bridge failed: {e.Message}");
    }
}
}
```

Pattern architetturale: soft dependency via reflection

Il bridge via reflection è il pattern raccomandato per mod che vogliono integrarsi opzionalmente con altre mod senza richiederle come hard dependency. Alternative: (a) [AttributeLoaded("..")] di Harmony per caricare conditionalmente le patch; (b) multimod target con LoadFolders.xml. Per [Mod Name] il bridge statico è sufficiente perché l'unica superficie di integrazione è un singolo metodo.

3. Schema Defs XML

Questa sezione presenta gli schemi XML delle Def principali. Per ciascuna Def sono mostrati gli attributi essenziali e i collegamenti alle classi C# introdotte nella Sezione 2. Gli snippet sono **rappresentativi ma non completi**: i valori concreti (costi esatti, stat numeriche, texPath) sono in Sezione 8 (Bilanciamento) e nei file di texture dedicati.

3.1 — ThingDef: Building_HybridGestator

La bay ibrida è definita come ThingDef che eredita da MaintenanceBayBase di Exosuit Framework. La classe C# custom (ModName.Building_HybridGestator) sostituisce la thingClass di default, e l'ITab_HybridBay viene registrato via inspectorTabs.

```
ModName_HybridGestator

hybrid gestator bay

Una stazione di gestazione ibrida che combina le funzioni
di Maintenance Bay (Exosuit Framework) e Mech Gestator (vanilla).
Permette di costruire mech-exosuit pilotati da subcore, configurarne
il loadout, ricaricarli e gestire l'installazione/estrazione del
subcore. Solo mech-exosuit possono essere ricaricati qui; i charger
vanilla e di altri mod non funzionano su di essi.
```

```
ModName.Building_HybridGestator

Things/Building/HybridGestator

Graphic_Multi
(4,4)

Things/Building/HybridGestator_Icon

250
8000
0.5
```

(3,3)
250
8
2
1
ModName_HybridMechtech
ModName.ITab_HybridBay
MF_Building_ComponentStorage
MF_Building_PayloadStorage
ToolCabinet
CompPowerTrader
350

```
ModName_MakePatriotMechExosuit
```

```
5
```

3.2 — PawnKindDef + ThingDef: Mech-Exosuit Race

Il mech-exosuit è un pawn che eredita da BaseMechanoid (vanilla) e monta i nostri ThingComp (CompSubcorePilot e CompMechExosuitBattery) via comps. Il BodyDef custom (Sezione 3.3) definisce il subcore come body part interna.

```
ModName_MechExosuitRace_Patriot
```

```
EX0-45 Patriot Mech-Exosuit
```

```
Una variante mech-piloted della Patriot Exosuit. Il  
pilota è un subcore Standard o High installato fisicamente come  
body part interna. Operativo fino a quando il subcore ha HP e la  
batteria ha energia.
```

```
ModName_MechExosuitBody
```

```
0
```

```
NULL
```

```
ToolUser
```

```
true
```

```
2000
```



```
0
```

```
Violent
```

```
Firefighter
```

```
Construction
```

```
Aqued_Exosuit_Apparel_Core
```

```
ModName_MechExosuit_Patriot
```

```
Patriot Mech-Exosuit
```

```
ModName_MechExosuitRace_Patriot
```

```
350
```

```
operative
```

```
true
```

3.3 — BodyDef: MechExosuitBody (con subcore come body part)

Il BodyDef custom è il cuore del sistema di danno. Definisce il subcore come body part interna con **coverage 0.10** (10% dei colpi che penetrano l'armatura colpiscono il subcore). Il subcore ha i suoi HP separati dall'HP del mech, gestiti dal CompSubcorePilot.

ModName_MechExosuitBody

```
mech-exosuit body
```

Torso

Middle

Outside

Torso

ModName_Subcore

subcore

0.10

Inside

ModName_SubcoreGroup

100

Head

0.10

Top

Eye

left sensor

0.05

Eye
right sensor
0.05

Shoulder
left shoulder
0.10

Arm
left arm
0.10

Shoulder
right shoulder
0.10

Arm
right arm
0.10

```
Leg
left leg
0.15
```

```
Leg
right leg
0.15
```

Perché coverage 0.10?

Coverage 0.10 significa che il 10% dei colpi che penetrano l'armatura colpiscono il subcore. Questo è un valore deliberatamente basso: il subcore è protetto dal frame dell'exosuit (lore: è nel torso, dietro la corazza). Un valore più alto (0.20+) renderebbe il sistema troppo punitivo — ogni raid danneggerebbe il subcore. 0.10 crea invece eventi rari ma significativi: il subcore viene colpito in media una volta ogni 10 colpi subiti, dando al giocatore tempo di ritirare il mech prima che sia troppo tardi.

3.4 — ThingDef: Relitto Sigillato + CompRelic

Il Relitto Sigillato è un item che dropa quando il mech-exosuit muore. Non può essere aperto istantaneamente: richiede un pawn che esegua il `JobDriver_DismantleRelic` (200 ticks di lavoro, equivalente a ~3 minuti di gioco). Il `CompRelic` conserva lo stato del subcore al momento della morte del mech (HP residui, tier, identità `SubcoreInfo`).

```
ModName_SealedRelic_Patriot

sealed relic of Patriot mech-exosuit

I resti contorti di un Patriot Mech-Exosuit distrutto.
```

Il relitto è sigillato; non è possibile sapere se il subcore interno è sopravvissuto finché non viene aperto. Lo smontaggio richiede lavoro manuale e produce: subcore recuperato (se sopravvissuto) o destroyed subcore, più materiali del frame.

Things/Item/SealedRelic_Patriot
Graphic_Single
(2,2)

100
80
400
0

ModName_Relics

false
None

ModName_DismantleRelic_Patriot
ModName_DestroyedSubcore_Standard
ModName_DestroyedSubcore_High

120
40
3

3.5 — ThingDef: Destroyed Subcore

Quando il subcore viene distrutto (HP = 0 in combattimento, o roll fallito all'apertura del relitto), dropa un **Destroyed Subcore**. Questo item non ha utilizzo diretto: può essere smontato al machining table per recuperare acciaio e componenti. Esistono due varianti (Standard/High) che differiscono solo nell'aspetto grafico e nei materiali resi.

```
ModName_DestroyedSubcore_Standard
```

```
destroyed standard subcore
```

```
Un subcore Standard andato irrimediabilmente distrutto.
```

```
Non è più utilizzabile come pilota per mech-exosuit. Può essere  
smontato al machining table per recuperare acciaio e componenti.
```

```
Things/Item/DestroyedSubcore_Standard
```

```
Graphic_Single
```

```
50
```

```
2
```

```
40
```

```
Items
```

```
Sellable
```

15
1

TableMachining

20
1

ModName_DestroyedSubcore_High
destroyed high subcore
Un subcore High andato irrimediabilmente distrutto...

Things/Item/DestroyedSubcore_High
Graphic_Single

50
3
80

Items

25

2

1

3.6 — RecipeDefs: crafting e smontaggio

Tre RecipeDef principali: (1) la ricetta di gestazione del mech (consuma subcore + materiali, eseguita nella bay ibrida); (2) la ricetta di smontaggio del relitto (eseguita dal pawn direttamente sul relitto, non in una bay); (3) la ricetta di smontaggio del Destroyed Subcore al machining table.

```
ModName_MakePatriotMechExosuit  
  
gestate Patriot mech-exosuit  
  
Gestisce un Patriot Mech-Exosuit usando un subcore  
Standard o High e i materiali del frame.  
  
Gestating Patriot mech-exosuit.  
  
60000  
  
8
```

SubcoreRegular

SubcoreHigh

1	
	Steel
150	
	Plasteel
250	
	ComponentIndustrial
10	
	SubcoreRegular
	SubcoreHigh
	Steel
	Plasteel

ComponentIndustrial

1

ModName_HybridGestator

ModName_HybridMechtech

ModName_DismantleRelic_Patriot

dismantle sealed relic

Apri il relitto sigillato per recuperare il subcore
(se sopravvissuto) e i materiali del frame.

Dismantling sealed relic.

200

GeneralLaborSpeed

4

Repair

Recipe_Machining

1

ModName.JobDriver_DismantleRelic

3.7 — Struttura directory del progetto mod

Organizzazione raccomandata per i file della mod, conforme alle convenzioni RimWorld 1.6 e al pattern di LoadFolders.xml:

```

ModName/
├─ About/
│   ├── About.xml          # metadati mod
│   ├── Preview.png
│   └─ PublishedFileId.txt
├─ LoadFolders.xml        # supporto 1.4/1.5/1.6
├─ Common/                # file condivisi tra versioni
│   ├── Defs/
│   │   ├── ThingDefs_HybridGestator.xml
│   │   ├── PawnKindDefs_MechExosuit.xml
│   │   ├── BodyDefs_MechExosuit.xml
│   │   ├── ThingDefs_Relics.xml
│   │   ├── ThingDefs_DestroyedSubcore.xml
│   │   ├── RecipeDefs_MechExosuit.xml
│   │   └─ ResearchDefs.xml
│   ├── Patches/
│   │   ├── Patch_ExosuitFramework.xml
│   │   ├── Patch_VanillaMechChargers.xml
│   │   └─ Patch_MechApparel.xml
│   ├── Textures/
│   │   ├── Things/Building/HybridGestator.png
│   │   ├── Things/Item/SealedRelic_Patriot.png
│   │   ├── Things/Item/DestroyedSubcore_Standard.png
│   │   ├── Things/Item/DestroyedSubcore_High.png
│   │   └─ Things/Pawn/MechExosuit_Patriot.png
│   └─ Languages/Italiano/Keyed/ModName.xml
├─ 1.6/
│   └─ Assemblies/

```

```

|   └─ ModName.dll           # compilato da sorgente C#
|   └─ Source/               # repo del sorgente C#
|   └─ ModName.sln
|   └─ ModName/
|       └─ CompSubcorePilot.cs
|       └─ CompMechExosuitBattery.cs
|       └─ Building_HybridGestator.cs
|       └─ ITab_HybridBay.cs
|       └─ JobDriver_DismantleRelic.cs
|       └─ CompRelic.cs
|       └─ SubcoreInfoBridge.cs
|       └─ HarmonyPatches/
|           └─ Patch_TakeDamage.cs
|           └─ Patch_MechDeath.cs
|               └─ Patch_MechCharger.cs
|       └─ ModSettings.cs
└─ CE/                       # Combat Extended compat (se caricato)
    └─ Patches/
        └─ Patch_MechExosuitCE.xml

```

CAPITOLO 04 · DANNO & MORTE

4. Sistema di Danno & Morte

Il sistema di danno di [Mod Name] è un **estensione del modello di Exosuit Framework**: il mech-exosuit ha un health pool principale (come qualsiasi pawn) e il subcore è trattato come body part interna con health pool separato. Quando un colpo penetra l'armatura e colpisce la body part 'Subcore', il danno viene applicato all'HP del subcore invece che all'HP del mech. Questo crea il flavor tematico 'il colpo ha trapassato la corazza e colpito il cervello del mech'.

4.1 — Flusso del danno (penetrazione al subcore)

Il diagramma sottostante illustra il flusso completo di un colpo ricevuto dal mech-exosuit, dalla generazione del DamageInfo fino all'esito finale (subcore illeso, subcore danneggiato, o subcore distrutto). Il sistema è **deterministico** in tutti i punti

tranne che nel roll di coverage (10% di probabilità che un colpo penetrante colpisca il subcore).

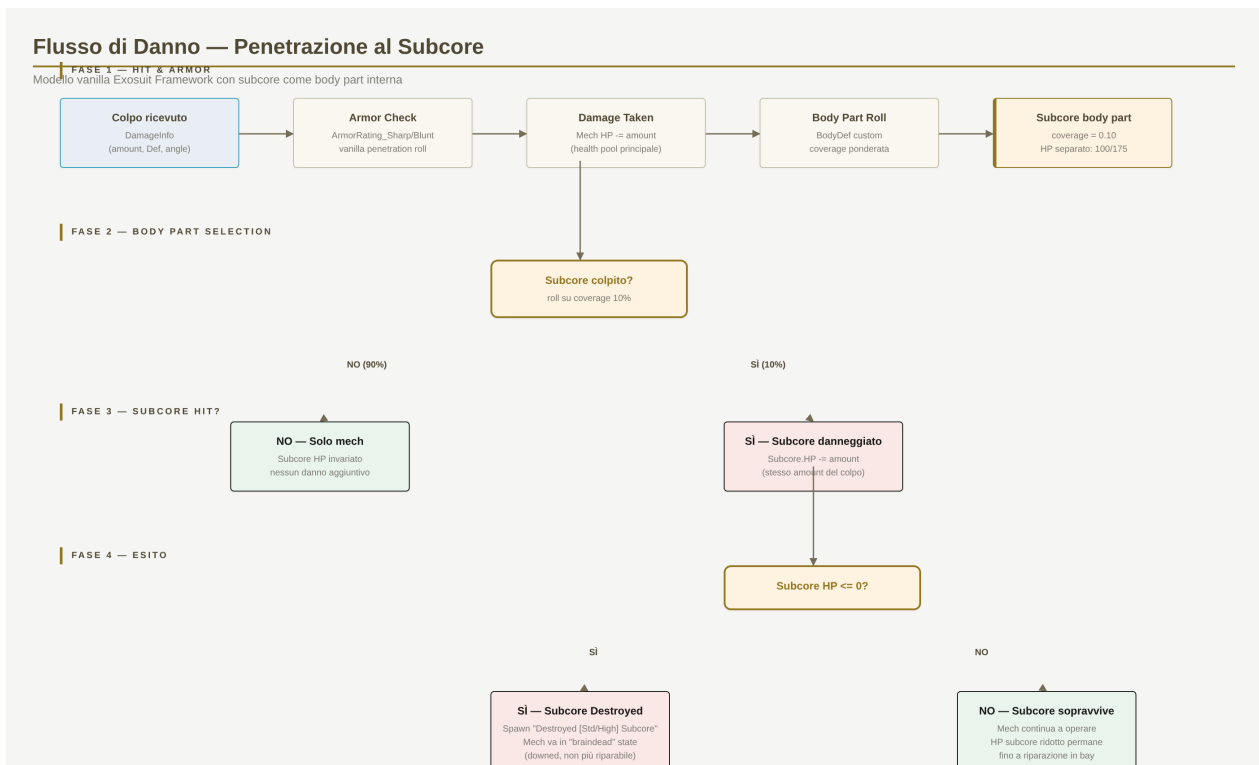


Figura 4.1 — Flusso del danno: colpo → armor check → body part roll → subcore hit (10%) → esito. Il subcore ha HP separato dal mech.

4.2 — Meccanica armor e body part selection

L'armor check è **completamente vanilla**: il mech-exosuit ha stat `ArmorRating_SSharp`, `ArmorRating_Blunt` e `ArmorRating_Heat` definite nel `ThingDef` (Sezione 3.2), e `RimWorld` gestisce il roll di penetrazione in modo standard. Se l'armatura blocca completamente il colpo, il danno è ridotto del 50% (mechanica Sharp armor) o del 100% (Blunt). Se l'armatura non blocca, il colpo penetra e procede al roll di body part.

La **body part selection** usa il `BodyDef` custom di Sezione 3.3. `RimWorld` sceglie la body part colpita in base alla coverage di ciascuna parte: Torso (35%), Head (10%), braccia (20% totali), gambe (30%), **Subcore (10%)**. Nota: coverage non è normalizzata a 100% — `RimWorld` fa un roll ponderato. Il subcore ha coverage 0.10 che si traduce in circa il 10% dei colpi penetranti che lo colpiscono.

4.2.1 — Quando il subcore viene colpito

Quando `RimWorld` determina che un colpo ha colpito la body part 'Subcore', viene chiamato `Pawn.TakeDamage` con un `DamageInfo` che ha `HitPart = Subcore`. Una Harmony patch nostra (vedi snippet sotto) intercetta questo caso e reindirizza il danno al `CompSubcorePilot` invece di applicarlo all'HP del mech. Questo preserva il flavor 'il colpo ha trapassato il frame ma non ha distrutto il mech — ha colpito il cervello'.

```
// HarmonyPatches/Patch_TakeDamage.cs
using HarmonyLib;

namespace ModName.HarmonyPatches
{
    [HarmonyPatch(typeof(Pawn), nameof(Pawn.TakeDamage))]
    public static class Patch_TakeDamage
    {
        public static bool Prefix(Pawn __instance, ref DamageInfo dinfo, ref
        DamageWorker.DamageResult __result)
        {
            // Solo per i nostri mech-exosuit
            if (__instance.def.defName.StartsWith("ModName_MechExosuitRace-"))
            {
                var subcoreComp = __instance.TryGetComp();
                if (subcoreComp == null || !subcoreComp.HasSubcore) return true;

                // Se il colpo ha colpito la body part Subcore, ridirigi
                if (dinfo.HitPart?.defName == "ModName_Subcore")
                {
                    subcoreComp.Notify_SubcoreHit(dinfo);
                    // Non applicare il danno all'HP del mech
                    __result = new DamageWorker.DamageResult();
                    return false; // skip vanilla damage application
                }
            }
            return true; // procedi con vanilla damage
        }
    }
}
```

4.3 — HP del subcore e distruzione

Tier subcore	HP massimi	Coverage body part	Note
Standard (SubcoreRegular)	100	0.10	Sblocca tier base. Richiede Standard Mechtech.
High (SubcoreHigh)	175	0.10	Tier avanzato. +75% HP, +4 skill bonus, 18h batteria.

Tabella 4.1 — HP del subcore per tier. Coverage identica perché dipende dal BodyDef.

Quando l'HP del subcore arriva a 0 (in combattimento, per colpi penetranti), si attiva `OnSubcoreDestroyed()` (Sezione 2.2.1). Effetti: (1) viene droppato un `Destroyed [Std/High]` Subcore come item separato accanto al mech; (2) il mech entra in stato 'braindead' — è downed, non può più operare, e non può essere riparato (perché il cervello è andato); (3) viene inviato un messaggio al giocatore con il nome del subcore (se `SubcoreInfo` è attivo) e la causa della distruzione.

Il mech in stato 'braindead' **non scompare**: rimane sul campo come 'shell vuota'. Il giocatore può: (a) abbandonarlo e lasciare che si deteriori naturalmente, (b) smontarlo al machining table per recuperarne i materiali del frame, (c) trascinarlo alla bay ibrida e — se ha un nuovo subcore — reinstallarlo per riattivarlo. L'opzione (c) è intenzionalmente costosa in lavoro (~4 ore di game) per bilanciare la possibilità di 'salvare' il frame.

4.4 — Flusso di morte del mech e drop del relitto

Quando l'HP **del mech** (non del subcore) arriva a 0, il mech muore. A differenza dei pawn umani (che droppano un corpo), il mech-exosuit dropa un **Relitto Sigillato**. Il relitto contiene potenzialmente il subcore ancora funzionante — ma il giocatore non lo sa finché non lo apre.

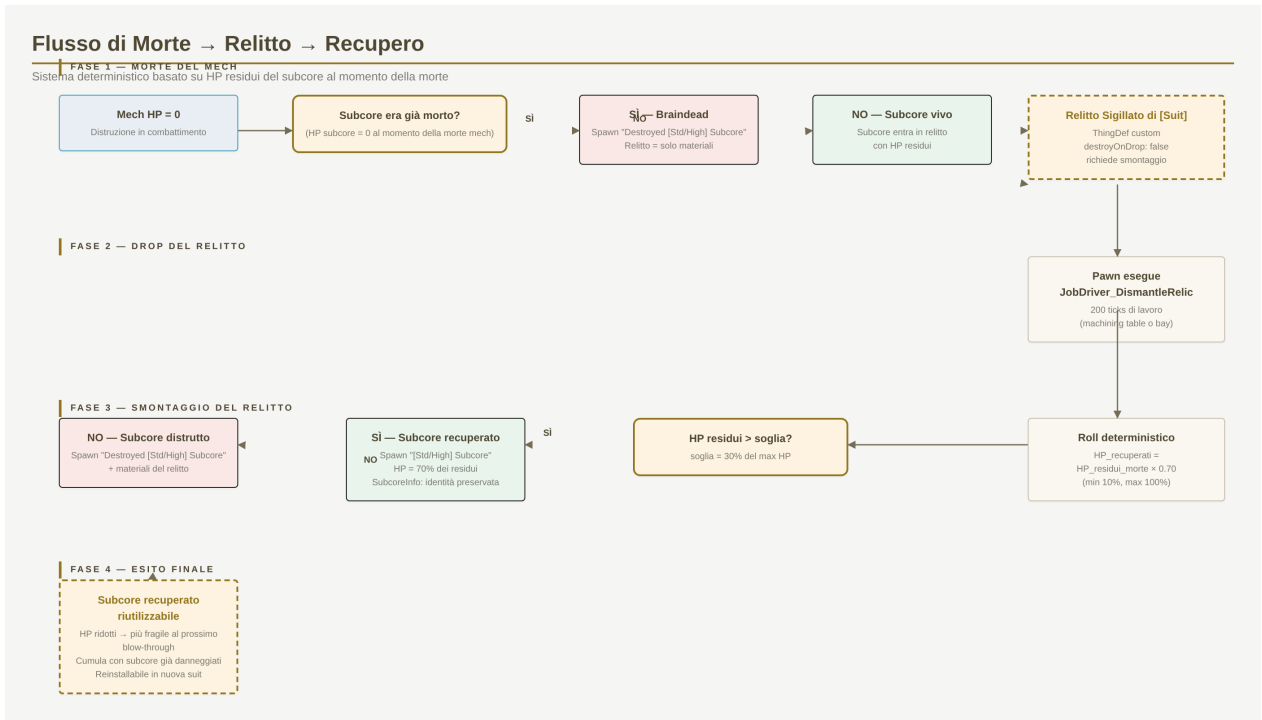


Figura 4.2 — Flusso di morte del mech, drop del relitto, smontaggio, esito finale. Il roll è deterministico in base agli HP residui del subcore.

4.5 — Formula di sopravvivenza del subcore

Il sistema di sopravvivenza del subcore nel relitto è **deterministico, non probabilistico**: non c'è un roll casuale. L'esito dipende esclusivamente dagli HP residui del subcore al momento della morte del mech. La formula:

Formula di recupero

$$\text{HP_recuperati} = \text{HP_residui_morte} \times 0.70$$

Soglia di sopravvivenza = 30% del max HP

Interpretazione: se il subcore aveva almeno il 30% dei suoi HP massimi al momento della morte del mech, sopravvive con HP ridotti al 70% dei residui. Se aveva meno del 30%, viene distrutto nell'esplosione del mech.

Esempio numerico per un subcore Standard (max 100 HP):

HP residui al momento della morte	Esito	HP subcore dopo recupero
100 (intatto)	Sopravvive	70 (= 100 × 0.70)
80	Sopravvive	56 (= 80 × 0.70)
50	Sopravvive	35 (= 50 × 0.70)
31 (appena sopra soglia)	Sopravvive	21 (= 31 × 0.70)

HP residui al momento della morte	Esito	HP subcore dopo recupero
30 (soglia)	Sopravvive	21 (= 30×0.70)
29 (appena sotto soglia)	Distrutto	— (spawn Destroyed Subcore)
0 (era già braindead)	Distrutto	— (spawn Destroyed Subcore)

Tabella 4.2 — Esempi numerici di esito del recupero. Soglia di sopravvivenza: 30 HP su 100 (Standard) o 52 HP su 175 (High).

4.6 — Degradazione cumulativa del subcore

Un subcore recuperato ha HP ridotti — ed è **riutilizzabile**. Ma quando viene reinstallato in un nuovo mech-exosuit, i suoi HP **non vengono ripristinati**: inizia già danneggiato. Questo significa che al prossimo blow-through, ha meno HP assoluti prima di essere distrutto. Inoltre, la soglia di sopravvivenza (30% del max) resta calcolata sul max originale — quindi un subcore con HP max 100 ma HP correnti 50 ha ancora bisogno di 30 HP residui per sopravvivere a un eventuale morte del mech.

Questo crea una **progressione di degrado**: ogni ciclo morte-recupero-riuso riduce gli HP del subcore di ~30% in media. Dopo 2-3 cicli, il subcore diventa troppo rischioso da usare in combattimento (soglia di sopravvivenza raggiunta rapidamente) e il giocatore è incentivato a smontarlo per i materiali. Questo è il **costo reale al fallimento** promesso dal Pilastro 1 della Sezione 1.

4.7 — Riparazione del subcore (opzionale)

Un subcore danneggiato **può essere riparato** alla bay ibrida usando un'operazione dedicata (costo: 2000 ticks di lavoro + 1 ComponentIndustrial + 20 Steel). La riparazione ripristina il subcore al suo max HP. Tuttavia, ogni riparazione riduce il MaxHP del subcore di 5 punti (irreversibile): dopo 5 riparazioni un subcore Standard passa da max 100 a max 75, rendendolo progressivamente meno attrattivo.

Bilanciamento: perché 5 HP per riparazione?

Il numero 5 è calcolato per rendere il break-even point intorno alla 5^a riparazione: a quel punto, il subcore ha max 75 HP, è più fragile di un nuovo Standard, e il costo cumulativo di riparazione ($5 \times 20 \text{ Steel} + 5 \times 1 \text{ ComponentIndustrial}$) è paragonabile al costo di creare un nuovo subcore. Il giocatore razionale smonta il subcore danneggiato e ne crea uno nuovo, chiudendo il loop di bilanciamento.

4.8 — Integrazione con Combat Extended

Combat Extended (CE) rimpiazza il sistema di danno vanilla con uno più realistico basato su **penetrazione armatura calcolata per colpo** anziché un roll binario. Per [Mod

Name], questo significa che il coverage del subcore (10%) va ricalcolato: in CE, il colpo 'sceglie' la body part dopo aver determinato se penetra l'armatura, ma la probabilità di colpire il subcore resta determinata dal BodyDef.

La compatibilità CE richiede una **Patch XML dedicata** in 1.6/CE/Patches/Patch_MechExosuitCE.xml che aggiunga ceBodyDef e BodyPartHealth custom per il subcore. Vedere Sezione 7 per il dettaglio.

CAPITOLO 05 · BAY & UI

5. Bay Ibrida & UI

La **Building_HybridGestator** è il cuore operativo della mod. È un edificio 3×3 che combina quattro funzioni precedentemente separate in RimWorld: il loadout editor di Exosuit Framework, la gestazione di mech vanilla, la stazione di ricarica per mech, e un'interfaccia per installazione/estrazione del subcore. Il risultato è un 'centro operativo mech-exosuit' che diventa il focal point della base del giocatore.

5.1 — Schema delle 4 funzioni integrate

Il diagramma sottostante illustra le quattro funzioni della bay ibrida, gli input che richiede (subcore + materiali + pawn crafter) e gli output che produce (mech-exosuit operativo, subcore recuperato su estrazione, frame item su estrazione).

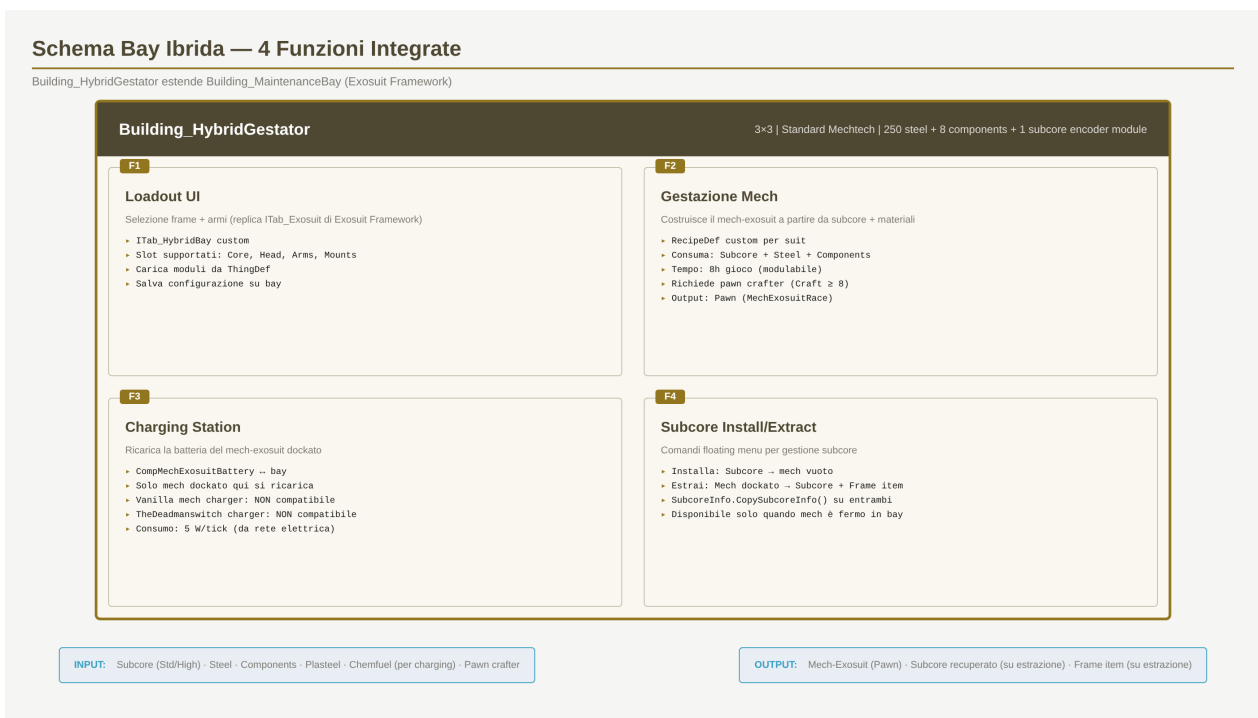


Figura 5.1 — Schema della Building_HybridGestator. Le 4 funzioni condividono lo stesso edificio 3×3 e lo stesso ITab.

5.2 — ITab_HybridBay: struttura della UI

L'ITab custom è il principale punto di interazione del giocatore con la bay. È diviso in **cinque sezioni verticali**, ciascuna dedicata a un aspetto operativo. La UI è ispirata a Exosuit.ITab_Exosuit ma è stata riscritta da zero (invece di subclassare) per evitare hard dependency sui membri internal della classe base.

Sezione UI	Funzione	Stati visualizzati
1. Header	Nome bay + stato operativo	Idle · Gestating · Charging · Subcore install pending
2. Loadout selector	Selezione frame + armi + moduli (replica ITab_Exosuit)	Lista slot supportati (Core, Head, Arms, Mounts); per slot: lista moduli disponibili
3. Subcore panel	Mostra subcore installato + HP + identità (se SubcoreInfo)	Nessun subcore · Subcore installato (Standard/High, HP/maxHP, nome pawn scansionato)
4. Battery status	Barra energia del mech dockato + tempo stimato alla carica	Nessun mech dockato · Charging (X%) · Fully charged
5. Gestation progress	Barra progressione della gestazione in corso	Nessuna gestazione · In corso (X%) · Completata

Tabella 5.1 — Le 5 sezioni dell'ITab_HybridBay. Scrollable se lo spazio non basta (440×540 pt di default).

5.3 — Loadout selector (replica ITab_Exosuit)

La sezione 2 (Loadout selector) replica il comportamento di Exosuit.ITab_Exosuit: per ogni slot supportato dal frame (definito nel SlotDef di Exosuit Framework), mostra il modulo attualmente installato e permette di cambiarlo con uno dei moduli disponibili nell'inventario della colonia. La differenza chiave: mentre ITab_Exosuit opera su un apparel indossato da un pawn, ITab_HybridBay opera sul mech-exosuit dockato.

I moduli supportati sono gli stessi di Exosuit Framework: per la Patriot, ad esempio, lo slot ArmRight supporta minigun o autocannon, lo slot MountLeft supporta rocket launcher. Quando il giocatore seleziona un modulo, l'ITab chiama Building_HybridGestator.TryInstallModule() che spawna un job per il pawn crafter di installarlo fisicamente.

5.4 — Workflow di gestazione

Il workflow di gestazione è il cuore della funzione F2:

- **Step 1** — Il giocatore clicca 'Avvia gestazione' nell'ITab. Appare un dialog con la lista dei mech-exosuit craftabili (in base alle ricette caricate dalla bay)
- **Step 2** — Selezione del mech-exosuit (es. Patriot). La bay verifica: subcore disponibile (in storage), materiali sufficienti, pawn crafter con Crafting ≥ 8 disponibile
- **Step 3** — Selezione del subcore da usare (se più di uno disponibile). La bay mostra il subcore e, se SubcoreInfo è attivo, il nome del pawn scansionato
- **Step 4** — Avvio. La bay consuma istantaneamente i materiali e il subcore (che viene 'installato' in una forma temporanea), e avvia la gestazione
- **Step 5** — Pawn crafter lavora alla bay per ~8 ore game (60.000 ticks di lavoro). La barra di progressione è visibile nell'ITab
- **Step 6** — Completamento. Spawn del mech-exosuit (Pawn) accanto alla bay. SubcoreInfoBridge.CopySubcoreInfo() trasferisce l'identità dal subcore al mech
- **Step 7** — Assegnazione al control group del mechanitor (se presente). Se nessun mechanitor con bandwidth disponibile, il mech rimane in stato 'unassigned' (downed, in attesa)

5.5 — Stazione di ricarica (F3)

Quando un mech-exosuit è dockato alla bay (cioè in piedi sulla cella di interazione della bay), la bay lo ricarica automaticamente. La ricarica richiede 350 W di potenza elettrica (consumo della bay in fase di charging) e procede a ~5 W/tick, il che significa che un mech-exosuit completamente scarico (0% energia) richiede ~3 ore di gioco per ricaricarsi al 100%.

Importante: i charger vanilla (MechCharger) e i charger di TheDeadmanswitch Mech Chargers **non funzionano** sui mech-exosuit. Questo è implementato via PatchOperationReplace che filtra i mech-exosuit fuori dal canCharge check di quegli edifici. Il flavor: i mech-exosuit hanno un sistema di ricarica proprietario compatibile solo con la bay ibrida.

```
/Defs/ThingDef[defName="MechCharger"]/comps/li[@Class="CompProperties_MechCharger"]/chargeableThings
```

```
Mech_Lifter
```

```
Mech_Constructoid
```

```
thedeadmanswitch.MechChargers
```

```
/Defs/ThingDef[starts-with(@ParentName,"TDS_MechChargerBase")]/comps/li[@Class="CompProperties_MechCharger"]/chargeableThings
```

5.6 — Installazione/estrazione subcore (F4)

I comandi floating per installazione/estrazione del subcore appaiono nel menu contestuale della bay (clic destro sulla bay con un pawn selezionato). Due operazioni:

Installa subcore

Precondizioni: (1) la bay ha un mech-exosuit 'shell vuota' (mech-exosuit senza subcore, creato gestando senza subcore — caso raro, o mech-exosuit braindead riportato alla bay); (2) il pawn selezionato ha un subcore (Standard o High) nell'inventario; (3) il pawn è adiacente alla bay. Effetto: il subcore viene 'installato' nel mech, che diventa operativo (se la batteria è carica).

Estrai subcore

Precondizioni: (1) la bay ha un mech-exosuit dockato con subcore installato; (2) il pawn è adadjente alla bay. Effetto: il subcore viene 'estratto' e droppato come item accanto alla bay. Il mech-exosuit torna 'shell vuota' (non operativo, può essere riusato installando un nuovo subcore). HP del subcore al momento dell'estrazione vengono preservati sull'item (così come l'identità SubcoreInfo).

Perché permettere estrazione?

L'estrazione volontaria serve a due scopi: (1) recuperare un subcore prezioso (ad esempio High) da un mech-exosuit che il giocatore non vuole più usare — per installarlo in un altro; (2) sostituire un subcore danneggiato con uno nuovo prima che venga distrutto in combattimento. Questo secondo caso è importante per il bilanciamento: permette al giocatore di "ritirare" un subcore storico prima che sia troppo tardi, ma il subcore resta danneggiato — non viene ripristinato dall'estrazione.

5.7 — Costo di costruzione e potenza

Voce	Valore	Note
Work to build	8.000 ticks (~1h game)	Equivalente a una fabrication bench
Costo materiali	250 Steel + 8 ComponentIndustrial + 2 ComponentSpacer + 1 SubcoreEncoderModule	SubcoreEncoderModule è vanilla 1.6
Max HP	250	Standard per edificio di produzione
Power consumption (idle)	50 W	Solo elettronica di base
Power consumption (charging)	350 W	Quando mech dockato in ricarica
Power consumption (gestating)	200 W	Durante gestazione attiva
Flammability	0.5	Più infiammabile di vanilla (ha componenti)
Research prereq	ModName_HybridMechtech	Sotto Standard Mechtech vanilla, vedi Sezione 9

Tabella 5.2 — Costi e consumi della Building_HybridGestator.

6. Batteria & Bandwidth

Due sistemi regolano l'**operatività** dei mech-exosuit: la **batteria** (energia temporanea, si scarica con l'uso) e la **bandwidth** (risorsa permanente del mechanitor, limita il numero di mech simultanei). Entrambi sono **deliberatamente scarsi**: il giocatore non può spammarli, deve pianificare quando e dove attivarli.

6.1 — Sistema batteria: design intenzionalmente scarso

Il CompMechExosuitBattery traccia l'energia del mech-exosuit come valore normalizzato 0..1, con consumo di 1/maxEnergyTicks per tick. Per default, maxEnergyTicks = 36.000 (~12 ore di gioco a velocità normale), ma il valore è configurabile per-tier via XML: i subcore High estendono la batteria a 54.000 ticks (~18 ore).

Tier subcore	maxEnergyTicks	~Durata gioco	~Durata reale (1x speed)	Note
Standard	36.000	12 ore	20 minuti	Mezza giornata di operatività
High	54.000	18 ore	30 minuti	Estensione del 50% rispetto a Standard

Tabella 6.1 — Capacità della batteria per tier. A velocità 3x (comune in late game), 12 ore gioco = ~7 minuti reali.

Quando l'energia arriva a 0, il mech va in **low power mode**: è downed (non può muoversi né attaccare), ma non è morto. Il giocatore riceve una notifica e un gizmo auto-attivato 'Ritorna alla bay' fa sì che il mech, appena diventa operativo (ad esempio per un pawn che lo ripara con chemfuel temporaneo), si muova verso la bay ibrida più vicina per ricaricarsi.

6.2 — Degradazione della batteria (meccanica opzionale)

Per aumentare la profondità tattica, la batteria può subire **degradazione cumulativa**: ogni ciclo completo di scarica-ricarica riduce la maxEnergyTicks del 2%. Dopo 10 cicli, il mech-exosuit ha perso il 20% della capacità originale. Questo costringe il giocatore a non lasciare il mech sempre acceso-spento-acceso: l'usura si accumula.

La degradazione può essere **riparata** alla bay ibrida con un'operazione dedicata (costo: 4.000 ticks + 1 ComponentIndustrial + 5 Chemfuel). Questo ripristina maxEnergyTicks al valore originale. La riparazione non ha costo in termini di maxHP del mech — è puramente una 'manutenzione' come cambiare l'olio.

6.3 — Routing della potenza elettrica

La bay ibrida ha tre modalità operative con consumi diversi:

Modalità	Consumo	Trigger
Idle	50 W	Bay costruita ma nessun mech dockato, nessuna gestazione in corso
Charging	350 W	Mech-exosuit dockato con batteria < 100%
Gestating	200 W	Gestazione attiva (indipendente dal charging)
Charging + Gestating	550 W	Entrambi in corso contemporaneamente (raro ma possibile)

Tabella 6.2 — Modalità operative della bay e relativi consumi.

Se la rete elettrica non riesce a fornire la potenza richiesta, la bay entra in **brownout**: la ricarica e la gestazione vengono messe in pausa (non annullate), e il giocatore riceve una notifica. Questo rende la bay sensibile alla qualità della rete elettrica della colonia, specialmente in late game quando la bay è solo una dei tanti carichi.

6.4 — Bandwidth del mechanitor

I mech-exosuit **consumano bandwidth del mechanitor** come mech vanilla. Entrano a pieno titolo nel **control group** assegnato, e rispondono ai comandi del mechanitor (draft, work, follow, escort). Se il mechanitor muore o perde bandwidth (ad esempio per overdose di mechlink), i mech-exosuit assegnati vanno in **dormant mode**: sono downed, non possono operare, ma non muoiono. Quando un nuovo mechanitor viene assegnato (o viene liberata bandwidth), tornano operativi.

6.4.1 — Costo in bandwidth

Il costo in bandwidth è **configurabile per suit** via XML (modExtension bandwidthCost). Per la Patriot, il valore di default è **3 bandwidth** — equivalente a un mini-mech vanilla come il Lifter, nonostante il mech-exosuit sia molto più potente. Questo è bilanciato dal fatto che il mech-exosuit ha costi aggiuntivi (batteria, subcore fragile) che i mech vanilla non hanno.

Tipo mech	Bandwidth cost	Note
Vanilla Lifter	1	Mini-mech, solo hauling
Vanilla Constructoid	2	Mini-mech, solo construction
Vanilla Pikeman	3	Combat mech
Vanilla Centipede	5	Heavy combat mech

Tipo mech	Bandwidth cost	Note
ModName Patriot Mech-Exosuit	3	Configurabile via XML; default uguale a Pikeman
ModName [altra suit]	2-5	Da definire per suit; vedi Sezione 8

Tabella 6.3 — Costo in bandwidth rispetto ai mech vanilla. Il mech-exosuit è bilanciato per essere un'alternativa ai combat mech vanilla.

6.5 — Integrazione con control group

I mech-exosuit usano il sistema vanilla di CompOverseerSubject per l'assegnazione al mechanitor. Quando un mech-exosuit viene generato dalla bay, viene automaticamente assegnato al control group del mechanitor più vicino con bandwidth disponibile. Se nessun mechanitor ha bandwidth, il mech rimane in stato 'unassigned' (downed) finché non viene liberata bandwidth.

Il giocatore può **cambiare control group** del mech-exosuit usando il gizmo standard 'Mech -> Assign to overseer' (vanilla). Quando il mech viene riassegnato, la bandwidth del vecchio mechanitor viene liberata e il nuovo mechanitor riceve il carico. Questo è importante se un mechanitor muore: il giocatore può riassegnare tutti i mech-exosuit a un altro mechanitor senza perderli.

6.6 — Sinergia batteria + bandwidth

I due sistemi creano **doppio costo opportunità**: anche quando un mech-exosuit è in low power mode (batteria scarica), **continua a consumare bandwidth**. Questo significa che il giocatore non può 'mettere in pausa' un mech-exosuit quando è scarico per liberare bandwidth — il costo bandwidth è fisso finché il mech esiste.

Perché questa sinergia è importante

Senza questa regola, il giocatore potrebbe fieldare 10 mech-exosuit e tenerli in pausa (scarichi) finché non servono, pagando solo il costo di crafting. Con la regola, ogni mech-exosuit ha un costo opportunità permanente in bandwidth: anche scarichi, occupano slot del mechanitor. Questo costringe il giocatore a smontare (estrarre subcore) i mech-exosuit che non usa più, invece di accumularli. È coerente con il Pilastro 2 (tensione operativa) della Sezione 1.

Il modo corretto di 'messa in pausa' di un mech-exosuit è **estrarre il subcore** (Sezione 5.6) e smontare il frame. Questo libera sia la bandwidth che il costo di mantenimento, ma costa lavoro e materiali per ricreare il frame in futuro. È un trade-off intenzionale.

CAPITOLO 07 · INTEGRAZIONI

7. Integrazione SubcoreInfo & Combat Extended

[Mod Name] è progettata con due integrazioni opzionali ma fortemente raccomandate: **SubcoreInfo** di eth0net (preserva l'identità del pawn scansionato sul subcore e la mostra nella scheda info del mech) e **Combat Extended** di NoImageAvailable (sistema di danno più realistico con penetrazione armatura calcolata per colpo). Entrambe le integrazioni sono **soft dependency**: la mod funziona anche senza, ma perde alcune feature.

7.1 — Integrazione con SubcoreInfo

SubcoreInfo è una mod che 'ricorda' quale pawn è stato scansionato per produrre un subcore. Funziona attachando un CompInfoBase agli scanner (softscanner, ripscanner) e al mech gestator vanilla, un CompSubcoreInfo ai subcore stessi (SubcoreRegular, SubcoreHigh), e un CompMechInfo a tutti i mech via BaseMechanoid. L'identità viene poi copiata automaticamente dal subcore al mech quando il mech viene generato.

Per [Mod Name], l'integrazione consiste in:

- **1. Trasferimento identità su gestazione** — quando la bay ibrida genera un mech-exosuit, chiamiamo `SubcoreInfoUtility.CopySubcoreInfo(subcoreItem, mechPawn)` per propagare l'identità del pawn scansionato dal subcore al mech-exosuit risultante
- **2. Trasferimento identità su recupero da relitto** — quando un pawn apre un Relitto Sigillato e il subcore sopravvive, chiamiamo `CopySubcoreInfo(relicThing, recoveredSubcoreItem)` per preservare l'identità sul subcore recuperato
- **3. Trasferimento identità su estrazione volontaria** — quando il giocatore estrae un subcore da un mech-exosuit dockato, chiamiamo `CopySubcoreInfo(mechPawn, subcoreItem)`
- **4. Patch su BaseMechanoid** — applichiamo la stessa patch di SubcoreInfo (`CompMechInfo` sui mech-exosuit) ma condizionata alla presenza di SubcoreInfo; se la mod non è caricata, la patch non viene applicata

7.1.1 — Chiamate al bridge

Tutte le chiamate passano per `SubcoreInfoBridge.CopySubcoreInfo` (Sezione 2.3), che è un no-op se SubcoreInfo non è caricato. Il bridge usa reflection per invocare il metodo pubblico di SubcoreInfo:

```
// SubcoreInfoBridge.cs (snippet, vedi 2.3 per completo)
```

```
public static void CopySubcoreInfo(Thing source, Thing dest)
{
    if (!IsLoaded) return;
    try
    {
        var asm = AppDomain.CurrentDomain.GetAssemblies()
            .FirstOrDefault(a => a.GetName().Name == "SubcoreInfo");
        var util = asm?.GetType("SubcoreInfo.SubcoreInfoUtility");
        var method = util?.GetMethod("CopySubcoreInfo",
            new[] { typeof(Thing), typeof(Thing) });
        method?.Invoke(null, new object[] { source, dest });
    }
    catch (System.Exception e)
    {
        Log.Warning($"[ModName] SubcoreInfo bridge failed: {e.Message}");
    }
}
```

7.1.2 — Quando il bridge viene chiamato

Evento	Source	Dest	Note
Gestazione mech-exosuit completata	Subcore item (consumato)	Mech-exosuit Pawn appena spawnato	In Building_Hybrid Gestator.CompleteGestation()
Apertura Relitto Sigillato (subcore sopravvissuto)	Relitto (Thing con CompRelic)	Subcore item spawnato	In JobDriver_DismantleRelic.openToil
Estrazione subcore (comando floating)	Mech-exosuit Pawn	Subcore item spawnato	In CompSubcorePilot.ExtractSubcore()
Installazione subcore (comando floating)	Subcore item (consumato)	Mech-exosuit Pawn	In CompSubcorePilot.InstallSubcore()

Tabella 7.1 — Tutti i punti in cui il bridge SubcoreInfo viene chiamato. In ogni caso, se SubcoreInfo non è caricato, la chiamata è no-op.

7.2 — Integrazione con Combat Extended

Combat Extended (CE) rimpiazza il sistema di danno vanilla con uno più realistico: ogni proiettile ha un valore di **penetrazione armatura (AP)** specifico, e l'armatura del bersaglio riduce il danno in modo proporzionale invece di un roll binario (bloccato/non bloccato). Questo ha implicazioni per il nostro sistema di subcore:

- **Il coverage del subcore resta 0.10** — la probabilità che un colpo penetrante colpisca il subcore non cambia con CE
- **La penetrazione al subcore è più realistica** — in CE, un colpo che penetra l'armatura del mech-exosuit ha già 'superato' la corazza; il subcore (che è dietro la corazza) viene colpito con il danno residuo
- **I proiettili ad alto AP sono più pericolosi per il subcore** — un proiettile antimateriale (AP 50+) che penetra ha quasi sempre abbastanza danno residuo da distruggere il subcore in un colpo
- **I proiettili a basso AP sono meno pericolosi** — un colpo di pistola (AP 5) che 'penetra' ha poco danno residuo e danneggia il subcore solo minimamente

7.2.1 — Patch CE dedicata

Per garantire compatibilità CE, [Mod Name] include una patch XML dedicata in 1.6/CE/Patches/Patch_MechExosuitCE.xml (caricata condizionalmente via LoadFolders.xml quando CE è presente). La patch fa tre cose:

```
/Defs/BodyDef[defName="ModName_MechExosuitBody"]
```

```
ModName_MechExosuitBody_CE
```

```
/Defs/ThingDef[starts-with(defName,"ModName_MechExosuitRace-")]/statBases
```

```
120
```

```
0.05
```

```
0.15
```

```
0
```

```
0.95
```

```
1.0
```

7.2.2 — BodyDef CE custom

CE usa un proprio sistema di BodyPartHealth per tracciare la salute delle singole body part. Per il subcore, definiamo una **body part custom CE** che ha un health separato e un coverage indipendente. La differenza chiave: in CE, la coverage può variare in base alla direzione del colpo (front/back/side), mentre in vanilla è uniforme. Per il subcore (nel torso), definiamo coverage 0.10 da tutte le direzioni tranne il dorso (0.15 — il subcore è più esposto da dietro).

7.3 — Altre compatibilità

Oltre a SubcoreInfo e CE, [Mod Name] è testata per compatibilità con:

Mod	Tipo dipendenza	Integrazione
Exosuit Framework (aoba.exosuit.framework)	Hard dependency	Necessaria per ParentName="MaintanenceBayBase" e per le Defs Slot/Module
Vanilla Expanded Framework (oskarpotocki.vfe.core)	Soft (via Exosuit Framework)	VEF è dipendenza di Exosuit Framework; usiamo ApparelExtension se presente
SubcoreInfo (eth0net.SubcoreInfo)	Soft dependency	Bridge via reflection, vedi 7.1
Combat Extended (ceteam.combatextended)	Soft dependency	Patch XML dedicata in 1.6/CE/, vedi 7.2
TheDeadmanswitch Mech Chargers	Compatibilità passiva	I nostri mech-exosuit sono esclusi dal charging di questi charger, vedi 5.5
Harmony (brrainz.harmony)	Hard dependency	Richiesta per le patch su Pawn.TakeDamage e altri
Vanilla Mechtech (Ludeon.RimWorld)	Hard dependency	Usa ThingDef SubcoreRegular, SubcoreHigh, BaseMechanoid, etc.

Mod	Tipo dipendenza	Integrazione
HugsLib	Opzionale, non richiesta	Nessuna integrazione; se caricata, non genera conflitti
Rocketman (kentington.saveourship2)	Opzionale, non richiesta	Compatibile (usa lo stesso sistema di caching vanilla)

Tabella 7.2 — Matrice di compatibilità con le mod più comuni.

7.4 — Matrice di testing raccomandata

Durante lo sviluppo, testare le seguenti combinazioni di mod per garantire che non ci siano regressioni:

- **Solo [Mod Name] + Exosuit Framework + VEF + Harmony** — baseline minima, deve funzionare
- **+ SubcoreInfo** — verifica che l'identità venga propagata in tutti e 4 i punti (Tabella 7.1)
- **+ Combat Extended** — verifica che il danno al subcore funzioni con penetrazione AP
- **+ SubcoreInfo + Combat Extended** — combinazione completa
- **+ TheDeadmanswitch Mech Chargers** — verifica che i mech-exosuit non vengano caricati dai charger custom
- **+ Vanilla Expanded: Mechanoids** — verifica compatibilità con mech custom aggiuntivi
- **+ Androids** — verifica che non ci siano conflitti con il sistema di pawn android (entrambi aggiungono pawnkind custom)

CAPITOLO 08 · BILANCIAMENTO

8. Bilanciamento — Caso Pilota Patriot

Questa sezione presenta i valori di bilanciamento **completi** per la Patriot Exosuit come caso pilota. Gli stessi valori (con adattamenti) si applicheranno alle altre suit quando verranno aggiunte (AMP Suit, Mobile Dragon, ecc.). I numeri sono **valori di partenza**, pronti per essere tweakati durante il playtesting.

8.1 — Stat del mech-exosuit (Patriot)

Le stat del mech-exosuit sono derivate direttamente dalla Patriot Exosuit originale di Aqued.Exosuits, con alcune modifiche per tener conto del fatto che non c'è un pawn

umano a bordo (niente stat offset che dipendono dal pawn).

Stat	Patriot originale	Patriot Mech-Exosuit	Razionale
MaxHitPoints (mech)	100 (apparel)	300 (pawn)	Pawn ha HP proprio + armor; +200 per durabilità
ArmorRating_Sharp	1.70	1.70	Invariato
ArmorRating_Blunt	1.80	1.80	Invariato
ArmorRating_Heat	1.50	1.50	Invariato
MoveSpeed	3.5 (offset)	3.5 (base)	Non dipende più da pawn Moving capacity
Mass	100	120	+20 per il frame di supporto mech
Insulation_Cold	60	60	Invariato
Insulation_Heat	40	40	Invariato
MarketValue	~1500 (apparel)	3500 (pawn)	Compreso subcore + materiali

Tabella 8.1 — Stat del Patriot Mech-Exosuit rispetto alla Patriot Exosuit originale.

8.2 — HP del subcore per tier

L'HP del subcore è **separato dall'HP del mech** ed è determinato dal tier del subcore installato. Standard ha 100 HP, High ha 175 HP. Questo valore è il 'punto di partenza' del subcore quando viene installato in un nuovo mech-exosuit.

Tier subcore	HP massimi	Soglia sopravvivenza (30%)	HP dopo 1° recupero (es.)	HP dopo 2° recupero (es.)
Standard (SubcoreRegular)	100	30	70 (da 100 residui × 0.7)	49 (da 70 residui × 0.7)
High (SubcoreHigh)	175	52	122 (da 175 residui × 0.7)	85 (da 122 residui × 0.7)

Tabella 8.2 — HP del subcore per tier e progressione di degrado. Dopo 2-3 recuperi, il subcore diventa troppo fragile per uso combat.

8.3 — Capacità della batteria

La capacità della batteria dipende dal tier del subcore: i subcore High forniscono più energia grazie alla migliore efficienza computazionale (lore: il subcore High gestisce meglio il power management).

Tier subcore	maxEnergyTicks	~Durata gioco	Ricarica completa	Consumo power bay
Standard	36.000	12 ore	~3 ore gioco (5 W/tick)	350 W
High	54.000	18 ore	~4.5 ore gioco (5 W/tick)	350 W

Tabella 8.3 — Capacità della batteria per tier. Il tempo di ricarica è lo stesso in entrambi i tier (5 W/tick).

8.4 — Costi di crafting

I costi di crafting del mech-exosuit includono i materiali del frame (identici alla Patriot originale) più il subcore (consumato) più componenti extra per il sistema di controllo mech.

Voce	Quantità	Note
SubcoreRegular (oppure SubcoreHigh)	1	Consumato alla gestazione, integrato come pilota
Steel	150	Struttura del frame (come Patriot originale)
Plasteel	250	Corazza del frame (come Patriot originale)
Uranium	50	Contropesi per gambe (come Patriot originale)
ComponentIndustrial	10 (+8 originali = 18 totali)	+2 per il sistema di controllo mech
ComponentSpacer	5	Per il sistema di controllo mech (come Patriot originale)
Work to make	60.000 ticks (~8h gioco)	Con 1 pawn crafter Crafting 8+
Skill requirements	Crafting 8	Come Patriot originale

Tabella 8.4 — Costo di gestazione del Patriot Mech-Exosuit. Il subcore è il componente più costoso (deve essere prodotto a parte).

8.5 — Lavori sbloccati per suit

I mech-exosuit **non possono fare qualsiasi lavoro**: per bilanciamento, ogni suit ha una lista definita di supportedJobs nel MechExosuitExt. La Patriot, essendo una suit da combat, supporta solo lavori violenti e di emergenza.

Job	Patriot Mech-Exosuit	Razionale
Violent (combat, hunting)	Sì	La Patriot è una suit da combat
Firefighter	Sì	Le suit meccaniche possono trasportare estintori
Construction	Sì (limitato)	Solo costruzioni base (muri, pavimenti); niente craft fine
Hauling	No	La Patriot non ha slot per cargo; usare mini-mech per hauling
Cleaning	No	Troppo grande per pulire interni
Growing	No	Calpesta le colture
Mining	No	Non ha slot per drill
Doctoring	No	Non ha manipolatori fini
Research	No	Non ha slot per computer
Art/Craft	No	Manipolatori troppo rozzi

Tabella 8.5 — Lavori supportati dalla Patriot Mech-Exosuit. Altre suit avranno liste diverse (es. Constructor Frame supporterà Construction e Hauling).

8.6 — Skill: sommatoria suit + subcore

Le skill del mech-exosuit sono calcolate come **sommatoria della skill base della suit + bonus del subcore**. La skill base della suit è definita nel MechExosuitExt (valori predefiniti per la Patriot: Combat 12, Construction 6). Il bonus del subcore è 0 per Standard, +4 per High.

Skill	Patriot base	Con Subcore Standard	Con Subcore High	Cap vanilla mech
Shooting	12	12	16 (cap 20)	8 (Pikeman)
Melee	8	8	12 (cap 20)	10 (Centipede)
Construction	6	6	10 (cap 20)	12 (Constructoid)
Mining	0	0	0	15 (Tunneler)
Cooking	0	0	0	0 (nessuno)
Crafting	0	0	0	0 (nessuno)
Animals	0	0	0	0 (nessuno)

Skill	Patriot base	Con Subcore Standard	Con Subcore High	Cap vanilla mech
Medicine	0	0	0	0 (nessuno)
Intellectual	0	0	0	0 (nessuno)

Tabella 8.6 — Skill del mech-exosuit. Con Subcore High, la Patriot supera il Pikeman vanilla in Shooting (16 vs 8). Bilanciato dal costo subcore High.

8.7 — Formula di recupero in pratica

La formula ($HP_{recuperati} = HP_{residui} \times 0.70$, soglia 30%) genera i seguenti scenari pratici per un subcore Standard (max 100):

Scenario	HP residui morte	Esito	HP dopo recupero	Cicli riusciti cumulativi
Subcore nuovo, mech muore subito	100	Sopravvive	70	1
Subcore 70 HP, mech muore	70	Sopravvive	49	2
Subcore 49 HP, mech muore	49	Sopravvive	34	3
Subcore 34 HP, mech muore	34	Sopravvive (sopra soglia 30)	24	4
Subcore 24 HP, mech muore	24	Distrutto (sotto soglia 30)	—	—

Tabella 8.7 — Progressione di degrado del subcore. Un subcore Standard può sopravvivere a ~4 cicli di morte-recupero prima di essere distrutto.

Per il subcore High (max 175), la progressione è simile ma estesa: il subcore sopravvive a ~5-6 cicli prima di essere distrutto. Questo rende i subcore High significativamente più valuable, giustificando il costo superiore di crafting.

8.8 — Altre suit: template di bilanciamento

Per le altre suit che verranno aggiunte (AMP Suit, Mobile Dragon, ecc.), applicare il seguente template:

- **Stat meccaniche:** partire dai valori della suit originale (armor, move speed, HP) e moltiplicare l'HP $\times 3$ per il passaggio da apparel a pawn
- **Costi crafting:** stessi materiali della suit originale + subcore + 2 ComponentIndustrial extra per il sistema di controllo mech

- **Skill base:** assegnare in base al ruolo (combat suit: Shooting/Melee alti; constructor suit: Construction alto; scout suit: MoveSpeed alto)
- **Jobs supportati:** in base al ruolo (combat: Violent+Firefighter; constructor: Construction+Hauling; scout: Violent+Cleaning)
- **Bandwidth cost:** 2-5 in base alla potenza (combat suit: 3-5; utility suit: 2-3)
- **Battery capacity:** 36.000 ticks per Standard, 54.000 per High (uniforme tra suit)

Bilanciamento iterativo

I valori in questa sezione sono **valori di partenza**, non finali. Playtesting continuo è essenziale: tenere traccia di quante volte un subcore viene distrutto, quanto dura un mech-exosuit in media, quanto spesso il giocatore usa l'estrazione volontaria vs smontaggio. Aggiornare i valori nel file XML MechExosuitExt della suit e ricompilare.

CAPITOLO 09 · TECH TREE

9. Albero Tecnologico

L'albero tecnologico di [Mod Name] è costruito **sopra** Standard Mechtech vanilla: il giocatore deve prima sbloccare Standard Mechtech (che dà accesso ai subcore Standard e al mechanitor), e solo dopo può ricercare i nodi nostri. Questo posizionamento è coerente con il design vanilla di mechtech come prerequisito per qualsiasi estensione mech.

9.1 — Visualizzazione dell'albero



Figura 9.1 — Albero tecnologico di [Mod Name]. Standard Mechtech (vanilla) è prerequisito diretto di HybridMechtech. I 3 nodi intermedi sono prerequisiti del nodo finale Master.

9.2 — Nodi di ricerca

Nodo	Costo (research)	Prerequisiti	Sblocca
ModName_HybridMechtech	4.000	Standard Mechtech (vanilla)	Building_HybridGestator + Patriot Mech-Exosuit (con Subcore Standard)
ModName_SubcoreFrameIntegration	4.000	ModName_HybridMechtech	Subcore HP +25% (Standard 125, High 220); sblocca riparazione subcore
ModName_AdvancedMechExosuit	6.000	ModName_HybridMechtech	Subcore High utilizzabile; +1 slot di loadout per tutte le suit
ModName_BatteryOptimization	3.000	ModName_HybridMechtech	Battery capacity $\times 1.5$ (Standard 18h, High 27h)
ModName_MasterMechExosuit	10.000	Tutti e 3 i nodi intermedi	Riparazione subcore senza perdita di maxHP; frame custom (scout, constructor)

Tabella 9.1 — Nodi di ricerca di [Mod Name]. Costo totale: 27.000 research points (~5-7 ore gioco per completare).

9.3 — Techprints

Per coerenza con il design vanilla di mechtech (che usa techprints per i nodi principali), anche i nodi di [Mod Name] richiedono **techprints**. I techprint sono item che riducono il costo di ricerca del 50% e sono fondamentali per rendere i nodi raggiungibili in tempo ragionevole.

Nodo	Techprints richiesti	Fonte techprint
ModName_HybridMechtech	2	Quest: "Mech-Exosuit Prototype" (rare quest reward)
ModName_SubcoreFrameIntegration	1	Trade: outlander towns (rare)
ModName_AdvancedMechExosuit	2	Quest: "Subcore High Recovery" (rare quest reward)
ModName_BatteryOptimization	1	Trade: outlander towns (uncommon)
ModName_MasterMechExosuit	3	Quest: "Ancient Mech Lab" (very rare, late game)

Tabella 9.2 — Techprints per nodo. I techprint sono item che dimezzano il costo di ricerca e sono ricercabili solo quando posseduti.

9.4 — Posizionamento nell'albero vanilla

I nodi di [Mod Name] si posizionano nel ramo **Mechtech** dell'albero vanilla, tra Standard Mechtech e High Mechtech. Il nodo finale (Master MechExosuit) è approssimativamente allo stesso livello di High Mechtech vanilla, rendendo il percorso di ricerca alternativo ma non sostitutivo: il giocatore che vuole usare mech-exosuit investe in [Mod Name], mentre il giocatore che preferisce mech vanilla investe in High Mechtech. Entrambi i percorsi possono essere percorsi in parallelo.

CAPITOLO 10 · ROADMAP

10. Roadmap MVP

Questa roadmap definisce le milestone di sviluppo di [Mod Name] in ordine di priorità e dipendenza. Ogni milestone ha una **definizione di done** chiara e un **set di task** ordinato. La roadmap è strutturata per permettere release incrementali: dopo M0 c'è già qualcosa di giocabile, anche se minimale.

10.1 — Panoramica milestone

ID	Nome	Stima ore	Dipenden ze	Definizione di done
M0	Hello World Mech	8-12	—	PawnKindDef custom spawna in gioco, è un mech vanilla funzionante
M1	Subcore Install	12-18	M0	CompSubcorePilot installato, HP tracking, SubcoreInfo bridge
M2	Death & Relic	16-24	M1	Drop relitto su morte, smontaggio, recupero subcore deterministico
M3	Bay Ibrida	20-30	M2	Edificio + ITab + 4 funzioni integrate
M4	Batteria & Bandwidth	10-15	M3	Batteria scarsa, low power mode, bandwidth cost, patch charger
M5	SubcoreInfo Integration	6-10	M3	Bridge via reflection testato in tutti i 4 punti (Tabella 7.1)
M6	CE Compat	8-12	M3	Patch CE dedicata, testing con CE caricata

ID	Nome	Stima ore	Dipendenze	Definizione di done
M7	Polish & Bilanciamento	20-40	M4, M5, M6	Playtesting, tweak valori, documentazione, pubblicazione

Tabella 10.1 — Milestone ordinate per dipendenza. Stima totale: 100-161 ore (~3-4 settimane full-time).

10.2 — M0: Hello World Mech

Obiettivo: avere un mech custom spawna in gioco usando le Defs minime. Questo valida l'ambiente di sviluppo (riferimenti DLL, compilazione, caricamento mod) e crea la base su cui costruire.

Task M0

- Setup progetto Visual Studio / Rider con riferimenti a Assembly-CSharp.dll, UnityEngine.dll, Exosuit.dll, 0MultiplayerAPI.dll
- About.xml minimo (packageId, name, supportedVersions, modDependencies per Exosuit Framework + Harmony)
- LoadFolders.xml per 1.6 (e opzionalmente 1.5)
- ThingDef race custom (ParentName="BaseMechanoid") con grafica placeholder
- PawnKindDef custom (ParentName="BaseMechanoidKind")
- Test: spawn via dev mode, verifica che il mech appaia e sia selezionabile

Done M0

Il mech custom spawna in gioco, ha grafica visibile (anche solo un rettangolo colorato), può essere selezionato e mostra la scheda info vanilla. Non ha behavior custom, è solo un mech vanilla con grafica diversa.

10.3 — M1: Subcore Install

Obiettivo: implementare il CompSubcorePilot in modo che un mech-exosuit possa 'ospitare' un subcore, tracciarne l'HP, e propagare l'identità via SubcoreInfoBridge.

Task M1

- Implementare CompSubcorePilot.cs + CompProperties_SubcorePilot.cs (Sezione 2.2.1)
- Aggiungere il Comp al ThingDef della race (Sezione 3.2)
- Implementare BodyDef custom con subcore come body part (Sezione 3.3)
- Implementare SubcoreInfoBridge.cs (Sezione 2.3) — solo stub per ora
- Implementare Harmony patch Patch_TakeDamage.cs per reindirizzare danno al subcore (Sezione 4.2.1)
- Test: spawnare mech-exosuit, installare subcore via dev mode, verificare HP tracking

- Test: SubcoreInfo installato, verificare che l'identità venga propagata

Done M1

Un mech-exosuit può essere generato con un subcore installato (via dev mode per ora), il subcore ha HP separati visibili nella scheda info, e quando il mech viene colpito alla body part 'Subcore', l'HP del subcore si riduce invece di quello del mech.

10.4 — M2: Death & Relic

Obiettivo: implementare il sistema di morte e drop del relitto, con smontaggio e recupero deterministico del subcore.

Task M2

- Definire ThingDef del Relitto Sigillato (Sezione 3.4) per la Patriot
- Definire ThingDef del Destroyed Subcore Standard e High (Sezione 3.5)
- Implementare CompRelic con logica CalculateSurvival() e CalculateRecoveredHP()
- Implementare JobDriver_DismantleRelic.cs (Sezione 2.2.5)
- Definire RecipeDef per smontaggio (Sezione 3.6)
- Implementare Harmony patch Patch_MechDeath.cs per intercettare la morte del mech e spawnare il relitto
- Aggiornare CompSubcorePilot con OnSubcoreDestroyed() e OnMechDeath()
- Test: uccidere mech-exosuit, verificare drop relitto, smontare, verificare esito deterministico
- Test: uccidere mech-exosuit con subcore a HP basso, verificare Destroyed Subcore

Done M2

Quando un mech-exosuit muore, dropa un Relitto Sigillato. Un pawn può smontare il relitto (200 ticks di lavoro) e ottenere: subcore recuperato con HP ridotti (se era sopra soglia) o Destroyed Subcore (se sotto soglia). SubcoreInfo propaga l'identità in entrambi i casi in cui il subcore sopravvive.

10.5 — M3: Bay Ibrida

Obiettivo: implementare la Building_HybridGestator con le 4 funzioni integrate (loadout, gestation, charging, subcore install/extract).

Task M3

- Definire ThingDef della Building_HybridGestator (Sezione 3.1)
- Implementare Building_HybridGestator.cs (Sezione 2.2.3)
- Implementare ITab_HybridBay.cs con le 5 sezioni UI (Sezione 5.2)
- Implementare HybridGestatorExt per le ricette supportate

- Definire RecipeDef per la gestazione della Patriot (Sezione 3.6)
- Implementare TryStartGestation() e CompleteGestation()
- Implementare charging tick (F3, Sezione 5.5)
- Implementare GetFloatMenuOptions per install/extract subcore (F4, Sezione 5.6)
- Patch XML per disabilitare i charger vanilla sui mech-exosuit (Sezione 5.5)
- Test: costruire bay, avviare gestazione, verificare spawn mech-exosuit
- Test: dockare mech-exosuit scarico, verificare ricarica
- Test: estrarre subcore, reinstallare, verificare HP preservati

Done M3

La bay ibrida è costruibile in gioco. Il giocatore può: costruire la bay, avviare la gestazione di un Patriot Mech-Exosuit (con Subcore Standard + materiali), ricaricare il mech-exosuit quando dockato, estrarre il subcore, reinstallarlo. L'ITab mostra tutte le 5 sezioni e funziona correttamente.

10.6 — M4: Batteria & Bandwidth

Obiettivo: implementare il sistema di batteria scarsa con low power mode, e l'integrazione con bandwidth del mechanitor.

Task M4

- Implementare CompMechExosuitBattery.cs (Sezione 2.2.2)
- Aggiungere il Comp al ThingDef della race
- Implementare CompTick con consumo energia
- Implementare Notify_LowPower() con downed state
- Implementare Recharge() e integrazione con la bay (Sezione 6.3)
- Verificare che il mech-exosuit entri in control group del mechanitor (vanilla CompOverseerSubject)
- Verificare che il mech-exosuit vada dormant quando il mechanitor muore
- Definire bandwidthCost nel MechExosuitExt (default 3 per Patriot)
- Test: fieldare mech-exosuit per 12 ore, verificare low power mode
- Test: ricaricare nella bay, verificare ripristino

10.7 — M5, M6, M7: integrazioni e polish

M5 — SubcoreInfo Integration

- Implementare tutti i 4 punti di chiamata al bridge (Tabella 7.1)
- Testare ciascun punto con SubcoreInfo caricata
- Testare ciascun punto senza SubcoreInfo (verifica no-op)

M6 — CE Compat

- Creare directory 1.6/CE/Patches/
- Scrivere Patch_MechExosuitCE.xml (Sezione 7.2.1)
- Definire BodyPartHealth CE per il subcore
- Testare con CE caricata: danno al subcore funziona con AP
- Testare senza CE: nessun regression

M7 — Polish & Bilanciamento

- Playtesting esteso: 3+ run di gioco complete con la mod
- Tweak valori di bilanciamento in base ai feedback (Sezione 8)
- Aggiungere texture custom per mech-exosuit, relitto, subcore distrutto
- Aggiungere supporto per altre suit (AMP Suit, Mobile Dragon) se richieste
- Scrivere Steam Workshop page (descrizione, screenshots, video)
- Pubblicazione v1.0

10.8 — Risk register (nota)

I principali rischi tecnici durante lo sviluppo, in ordine di probabilità:

- **R1 — BodyDef custom non interagisce bene con CE:** mitigazione = testare con CE fin da M1, non aspettare M6
- **R2 — Harmony patch su TakeDamage conflitto con altre mod:** mitigazione = usare prefix patch non invasive, priorità bassa
- **R3 — SubcoreInfoBridge reflection fallisce in versioni future di SubcoreInfo:** mitigazione = catch eccezioni, fallback a no-op con log warning
- **R4 — ITab custom conflitto con ITab_Exosuit di Exosuit Framework:** mitigazione = usare tab name unico, evitare di subclassare ITab_Exosuit
- **R5 — Performance: troppi mech-exosuit causano lag:** mitigazione = testare con 10+ mech-exosuit in late game, ottimizzare CompTick se necessario

CAPITOLO 11 · CHECKLIST

11. Checklist Implementazione

Questa sezione è pensata come **cookbook da tenere aperto** durante il coding. Per ogni milestone, fornisce: l'ordine operativo dei task, i riferimenti alle sezioni del documento, le signature API da implementare, e i pattern di testing.

11.1 — Setup iniziale del progetto

Prima di toccare qualsiasi codice C#, configurare l'ambiente:

- 1. Creare repo git: `git init ModName && cd ModName`
- 2. Creare struttura directory (Sezione 3.7)
- 3. Aggiungere come reference: Assembly-CSharp.dll (da RimWorld install), UnityEngine.dll, 0Harmony.dll, Exosuit.dll (da Exosuit Framework), 0MultiplayerAPI.dll (da Exosuit Framework)
- 4. Creare file About.xml con `packageId`, `name`, `supportedVersions=1.6`, `modDependencies` per Exosuit Framework + Harmony
- 5. Creare file LoadFolders.xml per supportare 1.6 (e opzionalmente 1.5)
- 6. Creare soluzione Visual Studio / Rider: ModName.csproj con target framework .NET Framework 4.8
- 7. Configurare build post-event: `copy /Y $(TargetDir)ModName.dll $(SolutionDir)..\1.6\Assemblies\`
- 8. Test: copiare la cartella mod in %RIMWORLD_DIR%/Mods/, lanciare gioco, verificare che la mod appaia nel mod manager

11.2 — API signatures pubbliche da implementare

Riepilogo delle API pubbliche delle 5 classi C#. Ogni signature deve essere implementata e testata individualmente prima di passare alla successiva.

Classe	Metodo pubblico	Firma	Sezione rif.
CompSubcorePilot	InstallSubcore	<code>void InstallSubcore(Thing subcoreItem)</code>	2.2.1
CompSubcorePilot	Notify_SubcoreHit	<code>void Notify_SubcoreHit(DamageInfo dinfo)</code>	2.2.1
CompSubcorePilot	OnMechDeath	<code>void OnMechDeath()</code>	2.2.1
CompSubcorePilot	ExtractSubcore	<code>void ExtractSubcore(Building_HybridGestator bay)</code>	2.2.1
CompMechExosuitBattery	Recharge	<code>void Recharge(float amount)</code>	2.2.2
CompMechExosuitBattery	Notify_LowPower	<code>void Notify_LowPower()</code>	2.2.2
Building_HybridGestator	TryStartGestation	<code>void TryStartGestation(RecipeDef recipe, Thing subcore)</code>	2.2.3
Building_HybridGestator	CompleteGestation	<code>private void CompleteGestation()</code>	2.2.3
Building_HybridGestator	GetFloatMenuOptions	<code>IEnumerable GetFloatMenuOptions(Pawn)</code>	2.2.3
ITab_HybridBay	FillTab	<code>protected override void FillTab()</code>	2.2.4

Classe	Metodo pubblico	Firma	Sezione rif.
JobDriver_Disman tleRelic	MakeNewToils	protected override IEnumerable MakeNewToils()	2.2.5
SubcoreInfoBridg e	CopySubcoreInfo (static)	static void CopySubcoreInfo(Thing src, Thing dst)	2.3

Tabella 11.1 — API pubbliche da implementare. L'ordine di implementazione segue le milestone M0-M7.

11.3 — Pattern di testing

Testing in RimWorld avviene principalmente **in gioco** via dev mode. Non esiste un framework di unit testing ufficiale, ma esistono pattern ripetibili per testare ciascuna feature.

11.3.1 — Test: danno al subcore

- Spawnare un mech-exosuit via dev mode (Action -> Spawn pawn -> ModName_MechExosuit_Patriot)
- Installare un subcore via dev mode (Action -> Set installed subcore)
- Aprire la scheda info del mech, verificare che il subcore sia visibile con HP corretti
- Spawnare un pawn hostile (es. raider) e forzarlo ad attaccare il mech
- Dopo 10-20 colpi penetranti, verificare che l'HP del subcore si sia ridotto
- Continuare fino a HP subcore = 0, verificare spawn Destroyed Subcore e mech in braindead state

11.3.2 — Test: morte del mech e drop relitto

- Spawnare mech-exosuit con subcore a HP vari (100, 50, 30, 20, 0)
- Uccidere il mech via dev mode (Action -> Damage -> instakill)
- Verificare spawn Relitto Sigillato
- Forzare un pawn a smontare il relitto (right-click -> Dismantle)
- Verificare esito in base alla tabella 4.2 (formula 30% soglia)

11.3.3 — Test: bay ibrida end-to-end

- Costruire la bay ibrida (Architect -> Production -> Hybrid Gestator Bay)
- Verificare che la bay appaia e abbia l'ITab custom
- Aprire l'ITab, verificare tutte le 5 sezioni
- Aggiungere materiali + subcore in una storage vicina
- Cliccare 'Avvia gestazione' nell'ITab, selezionare Patriot
- Verificare che un pawn crafter inizi a lavorare alla bay
- Attendere 8 ore game (~5 minuti reali a 3x speed)
- Verificare spawn del mech-exosuit + assegnazione al mechanitor

- Verificare che il mech-exosuit abbia il subcore installato (scheda info)

11.3.4 — Test: SubcoreInfo integration

- Installare SubcoreInfo mod
- Produrre un subcore via ripscanner (pawn scansionato: es. 'Colonist John')
- Verificare che il subcore mostri 'Origin: Colonist John' nella scheda info
- Usare il subcore per gestire un mech-exosuit
- Verificare che il mech-exosuit mostri 'Pilot: Colonist John' nella scheda info
- Uccidere il mech, aprire il relitto, verificare che il subcore recuperato mantenga 'Origin: Colonist John'

11.3.5 — Test: CE compat

- Installare Combat Extended mod
- Spawnare un mech-exosuit + un pawn hostile con arma ad alto AP (es. sniper rifle)
- Verificare che i colpi penetranti danneggino il subcore con danno residuo (non full damage)
- Verificare che i colpi a basso AP (es. pistol) non danneggino quasi mai il subcore

11.4 — Pitfall comuni e soluzioni

Pitfall	Sintomo	Soluzione
BodyDef custom non caricato	Mech-exosuit spawna senza body part subcore	Verificare che BodyDef sia in Defs/ e che defName nel ThingDef race match
Harmony patch non applicata	Danno al subcore non funziona	Verificare [HarmonyPatch] attribute + classe pubblica + ModName.cs con Harmony.PatchAll()
SubcoreInfo bridge fallisce silenziosamente	Identità non propagata ma nessun errore in log	Aggiungere Log.Message nel catch del bridge per debug
ITab non appare	Bay costruita ma senza ITab custom	Verificare inspectorTabs nel ThingDef e che ITab_HybridBay sia pubblica
Mech-exosuit non entra in control group	Mech spawnato ma rimane unassigned	Verificare CompProperties_OverseerSubject nel ThingDef e che mechanitor abbia bandwidth
Charger vanilla cerca di caricare mech-exosuit	Error spam in log quando mech-exosuit vicino a charger	Verificare PatchOperationReplace su chargeableThings del MechCharger
Relitto non droppa	Mech muore ma niente item a terra	Verificare Patch_MechDeath + che OnMechDeath sia chiamato (Log.Message di debug)

Pitfall	Sintomo	Soluzione
Subcore HP non serializzato	Save/load resetta HP subcore a max	Verificare PostExposeData in CompSubcorePilot con Scribe_Values.Look

Tabella 11.2 — Pitfall comuni durante lo sviluppo e relative soluzioni.

11.5 — Build e deploy

Workflow di build e deploy iterativo durante lo sviluppo:

- 1. Compilare in Visual Studio/Rider (Release config)
- 2. Copiare ModName.dll da bin/Release/ a 1.6/Assemblies/
- 3. Copiare l'intera cartella mod in %RIMWORLD_DIR%/Mods/ModName/
- 4. Lanciare RimWorld, attivare la mod nel mod manager
- 5. Testare in gioco
- 6. Per modifiche XML/Defs: chiudere il gioco, modificare i file, riaprire (non serve ricompilare C#)
- 7. Per modifiche C#: chiudere il gioco, ricompilare, copiare la DLL, riaprire
- 8. Per debug veloce: usare Log.Message() nel codice e dev mode -> log window

CAPITOLO 12 · GLOSSARIO

12. Glossario

Definizioni dei termini chiave usati nel documento e nella mod. I termini vanilla RimWorld sono marcati con **[vanilla]**, i termini di mod di terze parti con **[mod name]**, i termini introdotti da [Mod Name] con **[Mod Name]**.

Termine	Tipo	Definizione
Subcore	vanilla	Oggetto (Thing) prodotto da softscanner o ripscanner. Esistono 3 tier: Basic, Standard, High. In [Mod Name] usiamo solo Standard e High.
SubcoreRegular	vanilla	ThingDef del subcore Standard. HP vanilla 50, usato come pilota per mech-exosuit con HP custom 100.
SubcoreHigh	vanilla	ThingDef del subcore High. HP vanilla 80, usato come pilota per mech-exosuit con HP custom 175. Bonus +4 a tutte le skill della suit.

Termine	Tipo	Definizione
Bandwidth	vanilla	Risorsa del mechanitor che limita il numero di mech assegnabili. Ogni mech ha un bandwidthCost (1-5). I mech-exosuit consumano bandwidth come mech vanilla.
Mechanitor	vanilla	Pawn con implantato un Mechlink. Può controllare mech via Overseer subject. Prerequisito per usare mech-exosuit.
Control group	vanilla	Gruppo di mech assegnati a un mechanitor. Il mechanitor può comandare il gruppo (work, follow, escort, draft). I mech-exosuit entrano nel control group.
Dormant mode	vanilla	Stato di un mech quando il suo mechanitor muore o perde bandwidth. Il mech è downed ma non morto; si riattiva quando viene assegnato a un nuovo mechanitor.
ITab	vanilla	Tab nell'inspector panel di una Thing (sinistra). Customizzabile via inspectorTabs nel ThingDef. Noi usiamo ITab_HybridBay.
ModExtension	vanilla	Classe C# custom che può essere attachata a una Def via XML . Permette di aggiungere campi custom alla Def senza subclassare. Noi usiamo MechExosuitExt e HybridGestatorExt.
ThingComp	vanilla	Componente runtime attachato a una Thing via XML . Permette di aggiungere behavior custom. Noi usiamo CompSubcorePilot, CompMechExosuitBattery, CompRelic.
CompProperties	vanilla	Classe C# che definisce i parametri configurabili di un ThingComp via XML. Esempio: CompProperties_SubcorePilot.
Harmony patch	mod	Libreria (brrainz.harmony) che permette di modificare metodi di classi vanilla senza subclassare. Usata per Patch_TakeDamage e Patch_MechDeath.
BodyDef	vanilla	Def che descrive la struttura anatomica di un pawn. Noi creiamo ModName_MechExosuitBody con subcore come body part interna.
PawnKindDef	vanilla	Def che descrive un tipo di pawn (es. lifter, centipede, raider). Noi creiamo ModName_MechExosuit_Patriot.
Coverage	vanilla	Probabilità relativa che un colpo penetrante colpisca una body part. Il subcore ha coverage 0.10 (10% dei colpi penetranti).

Termine	Tipo	Definizione
Blow-through	vanilla	Meccanica per cui un colpo che distrugge una body part può passare alla body part adiacente. Rilevante per il danno al subcore.
Exosuit Framework	mod (aoba.exosuit.framework)	Mod di Aoba che fornisce il sistema di slot per exosuit (Core, Head, Arms, Mounts). Hard dependency di [Mod Name].
Exosuit.SlotDef	mod (Exosuit Framework)	Def custom di Exosuit Framework che definisce gli slot di equipaggiamento di una exosuit. 7 slot: Core, Head, Attachment, ArmLeft, ArmRight, MountLeft, MountRight.
Exosuit.Exosuit_Core	mod (Exosuit Framework)	Classe C# base dell'apparel-exosuit. Le nostre mech-exosuit sono pawn (non apparel), ma usano lo stesso pattern di danno.
Building_MaintenanceBay	mod (Exosuit Framework)	Classe C# della docking bay originale di Exosuit Framework. La nostra Building_HybridGestator la estende.
IExosuitDestructionHandler	mod (Exosuit Framework)	Interfaccia pubblica di Exosuit Framework per intercettare la distruzione di una exosuit. Noi la implementiamo per drop il relitto.
SubcoreInfo	mod (eth0net.SubcoreInfo)	Mod di eth0net che traccia quale pawn è stato scansionato per produrre un subcore. Soft dependency di [Mod Name].
SubcoreInfoUtility.CopySubcoreInfo	mod (SubcoreInfo)	Metodo pubblico statico di SubcoreInfo che copia l'identità del pawn scansionato da un Thing a un altro. Usato dal nostro bridge.
Combat Extended (CE)	mod (ceteam.combatextended)	Mod di CTeam che rimpiazza il sistema di danno vanilla con uno più realistico basato su AP (armor penetration). Soft dependency di [Mod Name].
Vanilla Expanded Framework (VEF)	mod (oskarpotocki.vfe.core)	Framework di Oskar Potocki usato da molte mod VE. Dipendenza di Exosuit Framework, soft per [Mod Name].
Mech-exosuit	[Mod Name]	Pawn custom generato dalla bay ibrida. È un mech vanilla a tutti gli effetti (consume bandwidth, va dormant) ma con subcore installato come body part interna.
Bay ibrida	[Mod Name]	Building_HybridGestator. Edificio 3×3 che combina 4 funzioni: loadout editor, gestation, charging, subcore install/extract.

Termine	Tipo	Definizione
Relitto Sigillato	[Mod Name]	Item droppato quando un mech-exosuit muore. Contiene potenzialmente il subcore. Richiede smontaggio (JobDriver_DismantleRelic) per rivelare l'esito.
Destroyed Subcore	[Mod Name]	Item droppato quando il subcore viene distrutto (HP = 0 in combattimento, o roll fallito all'apertura del relitto). Smontabile al machining table per acciaio e componenti.
Subcore braindead	[Mod Name]	Stato del mech-exosuit quando il suo subcore viene distrutto in combattimento. Il mech è downed e non riparabile, ma può essere smontato per i materiali o riportato alla bay per reinstallare un nuovo subcore.
Low power mode	[Mod Name]	Stato del mech-exosuit quando la batteria arriva a 0. Il mech è downed ma non morto. Si ripristina ricaricandolo nella bay ibrida.
Shell vuota	[Mod Name]	Mech-exosuit senza subcore installato. Può essere creato gestando senza subcore (raro), o essere il risultato di estrazione volontaria o subcore braindead. Non operativo finché non si installa un nuovo subcore.
Soft dependency	[Mod Name] pattern	Pattern in cui [Mod Name] funziona anche se la mod dipendente non è caricata, ma perde alcune feature. Implementato via ModDetector + reflection bridge.
Hard dependency	[Mod Name] pattern	Pattern in cui [Mod Name] non può funzionare senza la mod dipendente. Implementato via modDependencies in About.xml. Exosuit Framework è hard dependency.

Tabella 12.1 — Glossario dei termini usati nel documento e nella mod.

Prossimi passi

Questo documento è la **baseline di design** per [Mod Name]. Il prossimo passo è implementare M0 (Hello World Mech) seguendo la checklist della Sezione 11. Una volta raggiunto M0, procedere con M1 (Subcore Install) e così via. Tornare a questo documento per rivedere le decisioni di design quando si implementano feature specifiche. Aggiornare il documento (nuova versione) quando cambiano le decisioni di bilanciamento o si aggiungono nuove suit.